从定制Workflow到AI自主决策的架构演进——以TMIC AI小新为例

原创 天猫新品营销技术 大淘宝技术 2026年5月29日 02:09 浙江 19人

本文分享了TMIC AI小新技术架构从**定制化workflow到DeepAgent模式的架构改版实践**。面对复杂问题需要跨模块协作、多步骤推理和动态参数识别的挑战，我们借鉴DeepAgent的核心设计思路（TodoList、SubAgent、Summary、FileSystem），实现了从"预设流程"到"AI自主决策"的转变。与此同时，我们还在DeepAgent的基础上针对业务场景做了**创新性优化（Tree Action模式、SubAgent提速、异步Summary）**，显著提升了系统性能和效率。

01 引言

产品介绍与业务价值

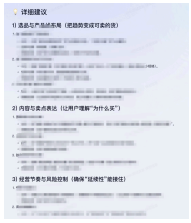
AI小新是TMIC平台的核心AI问答产品，其核心功能是通过问答的方式，让用户能够利用TMIC平台的数据，快速进行数据洞察。用户只需通过自然语言提问，即可获取所需的数据分析和业务建议，无需深入了解平台的复杂功能模块。

AI小新产品的核心价值在于通过问答的方式，让用户能够利用TMIC平台的数据，快速进行数据洞察，降低商家使用平台的门槛。具体体现在：

1. **降低使用门槛**：商家无需深入了解平台的复杂功能模块，只需通过自然语言提问即可获取所需的数据和洞察
2. **整合业务模块**：帮助商家对平台内众多业务模块进行整合串联，实现跨模块的数据获取和综合分析
3. **提升决策效率**：通过AI智能分析，快速为商家提供新品研发建议、市场洞察等业务建议

随着产品的不断迭代，我们发现系统对于具有一定确定性的问题回答效果尚可，但仍然缺乏灵活性，难以应对复杂和开放性问题。

产品示例图：

TMIC 各个数据产品	分析结论	分析过程	详细建议
站内趋势： AI行业趋势报告： ……省略		 ……省略	

面临的挑战

随着产品的不断迭代，我们发现系统对于具有一定确定性的问题回答效果尚可，但仍然缺乏灵活性，难以应对复杂和开放性问题。问题的核心在于：**从简单问题到复杂问题的挑战**。

在业务实践中，商家的问题呈现出明显的两极分化：一方面，大量简单问题（如"某类目某时间段的销售额是多少"）可以通过预设的workflow直接处理，系统表现稳定可靠；另一方面，越来越多的复杂问题（如"帮我分析适合研发哪类新品并分析竞品特点"）需要跨模块协作、多步骤推理和动态参数识别，现有架构却无法有效处理。

这种挑战不仅限制了产品的应用场景，也制约了AI能力的充分发挥。如何让系统既能稳定处理简单问题，又能灵活应对复杂问题，成为架构改版的核心目标。

- 简单问题：当前可较好处理

简单问题具有明确的特征：

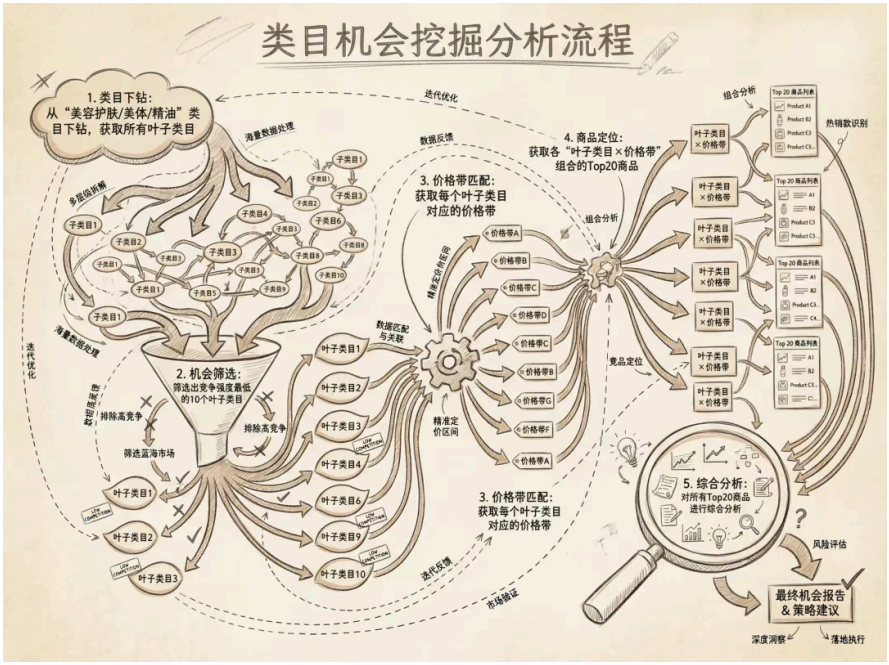
- **特征**：具有明确的取数意图和分析意图，能够直接对应到产品上的特定数据集合
- **示例**：
  - "2025年12月女装连衣裙类目下销售额为多少？"
  - "2025年12月女装连衣裙类目的不同价格带表现如何？"

这类问题的特点是参数明确、意图单一、数据来源固定，系统能够通过预设的workflow直接处理。

- 复杂问题：当前处理困难

复杂问题则呈现出完全不同的特征：

- **特征**：开放创造性问题，需要跨模块协作、多步骤推理、动态参数识别
- **示例**：
  - "我想研发美容护肤类的新品，请帮我分析一下我适合研发哪类新品（选10个竞争强度最低的叶子类目），并帮我分析一下我可能对标的竞品（这些机会类目对应的Top20竞品）有什么特点？"



这类问题的理想分析路径通常包括多个阶段，需要系统具备自主规划、动态参数识别和跨模块协作的能力，核心特征在于层层下钻：

阶段一：类目层层下钻与机会筛选

1. 在类目趋势报告中获取美容护肤/美体/精油(1801)类目下的子类目
2. 层层下钻：从一级类目开始，逐层下钻到二级、三级...直到获取所有叶子类目（类目数量可能达到数百个，参数需要动态获取，无法提前确定）
3. 筛选机会类目：从众多叶子类目中，基于竞争强度等指标进行筛选，筛选出竞争强度最低的10个叶子类目作为机会类目

阶段二：价格带下钻与商品下钻

4. 下钻到价格带维度：在新品管理模块，针对筛选出的10个机会类目，分别获取每个类目对应的价格带信息（跨模块协作，每个类目可能有多个价格带）
5. 下钻到商品维度：对于每个叶子类目×价格带组合，进一步下钻获取该组合下的Top20商品数据（10个类目 × 多个价格带 × Top20商品，涉及上百次工具调用）

阶段三：综合分析

6. 对所有Top20商品进行综合分析，提炼竞品特点、定价策略、产品利益点等共性特征，并给出新品研发建议

执行复杂度：此类问题涉及上百次工具调用，需要跨多个业务模块协作，且参数识别需要动态下钻获取，无法提前确定。

- 理想分析路径 vs 实际处理能力的差距

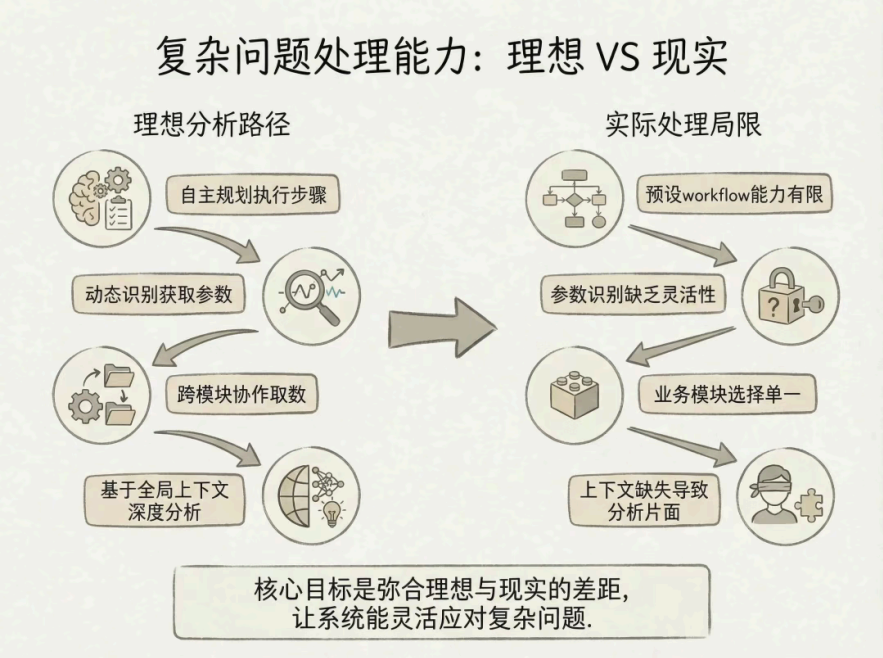
面对复杂问题，理想情况下系统应该能够：

1. **自主规划执行步骤**：系统能够理解问题的复杂性，自主拆解任务，规划出清晰的执行路径。
2. **动态识别和获取参数**：系统能够根据执行过程中的中间结果，动态识别下一步需要的参数，而不是依赖预设的参数识别流程。
3. **跨模块协作获取数据**：系统能够灵活调用多个业务模块的工具，实现跨模块的数据获取和整合分析。
4. **基于全局上下文进行深度分析**：系统能够基于完整的执行历史、取数过程和中间上下文信息，进行深度分析和综合洞察。

但实际情况是，定制化workflow架构存在明显的局限性：

1. **自主规划能力有限**：系统只能按照预设的workflow执行，当遇到超出预设workflow的问题时，无法自主调整执行路径，也无法应对各种边界情况。
2. **参数识别缺乏灵活性**：部分参数的逻辑依赖预设的workflow，缺乏灵活性。
3. **业务模块选择固定单一**：系统只能选择一个业务模块，无法跨模块协作。
4. **上下文信息缺失**：最后生成结论时依赖的上下文信息有限，无法感知全局思考过程，导致分析结果缺乏深度和全局视角。

这种理想分析路径 vs 实际处理能力的差距，严重制约了系统处理复杂问题的能力。如何让系统既能稳定处理简单问题，又能灵活应对复杂问题，成为架构改版的核心目标。

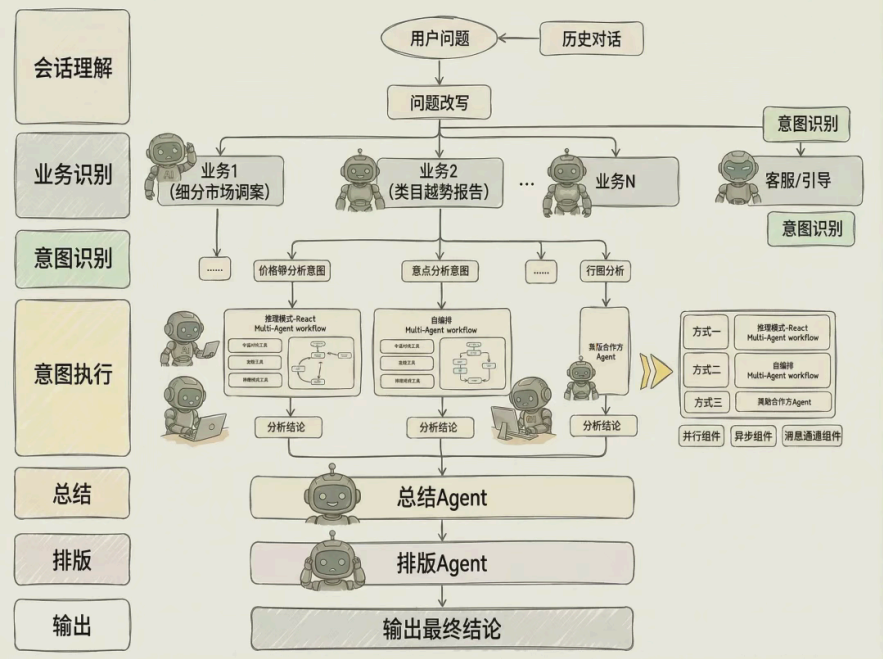


02

Ai小新的演进之路：从定制化Workflow到AI自主决策（DeepAgent）

定制化workflow的解决方案与痛点

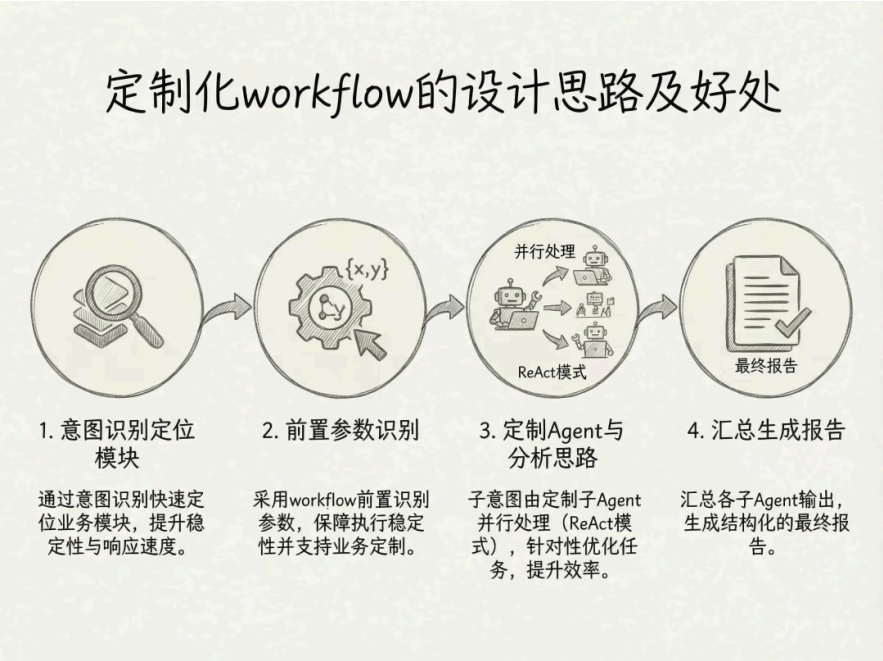
- 定制化workflow如何解决问题



核心流程设计：

1. **会话理解**：结合当前对话和历史对话，对用户的问题进行改写，改写后的问题应当是一个能够完整表达用户意图的语句
2. **业务识别**：根据用户的问题去定位到由哪个业务模块去承接
3. **意图识别**：每个业务下可能会有众多的取数/分析意图，判断用户的问题可能会命中哪些意图（可能同时命中多个意图）
4. **意图执行**：真正进行取数/分析的地方，每个意图背后通常都是一个Multi-Agent workflow，根据意图本身的特点可以选择不同的workflow模版，也可以直接接第三方的外部服务。为了提升执行速度，这里多个意图采取的是并行的执行方式。
5. **总结**：对多个意图执行结果进行整合、总结，生成最终的回答结论
6. **排版**：对最终结果进行排版，以适用于前端展示

定制化workflow的设计思路及好处：



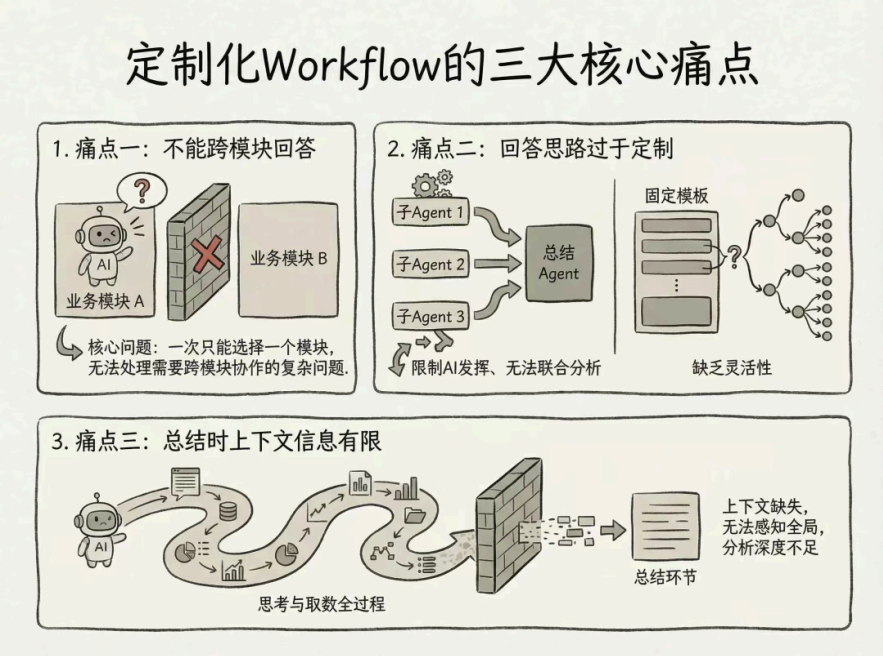
1. **意图识别定位业务模块**
   - 设计思路：通过业务识别、功能识别等意图识别方式，快速准确定位到业务模块。
   - 好处：提升稳定性，确保能够准确定位到对应业务模块；缩短响应时间。如果识别不到关键参数可以快速退出。
2. **前置参数识别**
   - 设计思路：通过workflow的方式对某些参数进行前置识别，支持业务逻辑定制。
   - 好处：保证后续执行的稳定性，避免在执行过程中出现参数错误，同时支持大量业务逻辑定制。
3. **定制Agent和定制分析思路**

- 设计思路：每个子意图由单独的子Agent负责，可以定制特定的分析思路和取数逻辑，子Agent内为ReAct模式，并行执行。
- 好处：针对性地优化每个子任务的执行效果，提升整体执行效率。

4. **汇总生成报告**
   - 设计思路：汇总各个定制子Agent的输出，生成结构化的最终报告。
   - 好处：生成结构化的最终报告。

- 定制化workflow的核心痛点

虽然定制化workflow带来了确定性和稳定性，但也严重限制了AI的自主发挥空间，存在以下三大核心痛点：



痛点1：不能跨产品模块回答

在此架构中，业务模块选择这里会定位一个确定的业务模块，因此在后续链路中也只能基于一个确定的业务模块去回答，一旦跨模块了就无法进行回答。

实际案例：复杂问题"我想研发美容护肤类的新品，请帮我分析一下我适合研发哪类新品"需要同时使用业务模块1（获取类目数据）和业务模块2（获取竞品数据），但定制化workflow只能选择一个业务模块，无法跨模块协作。

痛点2：回答思路过于定制

定制化的workflow虽然带来确定性，但严重限制了AI的自主发挥空间。主要体现在两个方面：

① 功能模块拆分定制

- **设计方式**：每个业务模块下预设功能模块，每个功能模块作为独立的子Agent，各子Agent并行执行，互不感知，由总结Agent进行最终汇总
- **存在的问题**：
  - 致命缺陷：提前为AI划分边界，限制了AI的自主发挥空间
  - 各子Agent执行过程相互独立，无法进行联合分析或下钻分析
  - 仅依赖总结Agent进行汇总，分析深度不足
  - 遇到超出预设workflow的问题时，系统无法处理

② **参数识别定制**

- **设计方式**：在业务模块识别后，执行定制化的参数识别workflow，必须从问题中定位到确定的参数
- **存在的问题**：
  - ❌ 缺乏灵活性：无法处理需要动态获取参数的问题
  - 例如：类目需要通过层层下钻才能确定，无法在问题中直接识别
  - 例如：用户想同时对比多份报告，但系统只支持单份报告识别

痛点3：总结时上下文信息有限，思考过程感知不全面

在总结阶段最终生成报告时，系统所依赖的上下文信息非常有限：

1. **上下文严重缺失**：
   - 总结时只能获取到有限的数据字段（如功能模块、单模块数据、数据描述、用户问题等）
   - 取数过程及中间产生的上下文信息未传递给总结环节
   - 无法感知全局思考过程，仅基于最终取数结果进行总结
   - 结果：总结效果片面，缺乏深度和全局视角
2. **无法跨模块综合分析**：
   - 总结环节嵌入在每个子Agent内部，各子Agent执行过程相互独立
   - 无法同时对多个模块的数据进行综合分析
   - 限制了跨模块洞察的生成

DeepAgent模式的解决方案

基于上述问题分析，新版架构旨在解决定制化workflow的核心矛盾，在保持稳定性的同时，大幅提升系统的灵活性和AI的自主能力。

- 设计目标

目标1：支持跨模块回答

- 支持在单次对话中调用多个业务模块的工具，实现跨模块数据获取和综合分析
- 解决定制化workflow"业务模块选择固定单一"的问题

目标2：支持更灵活的取数方式

- 将预设workflow转交给AI自主决策，充分发挥大模型的规划和分析能力
- 支持动态参数识别（如通过下钻获取类目）、多参数组合和多报告对比等灵活场景
- 解决定制化workflow"参数识别定制"和"功能模块拆分定制"的问题

目标3：丰富分析上下文信息

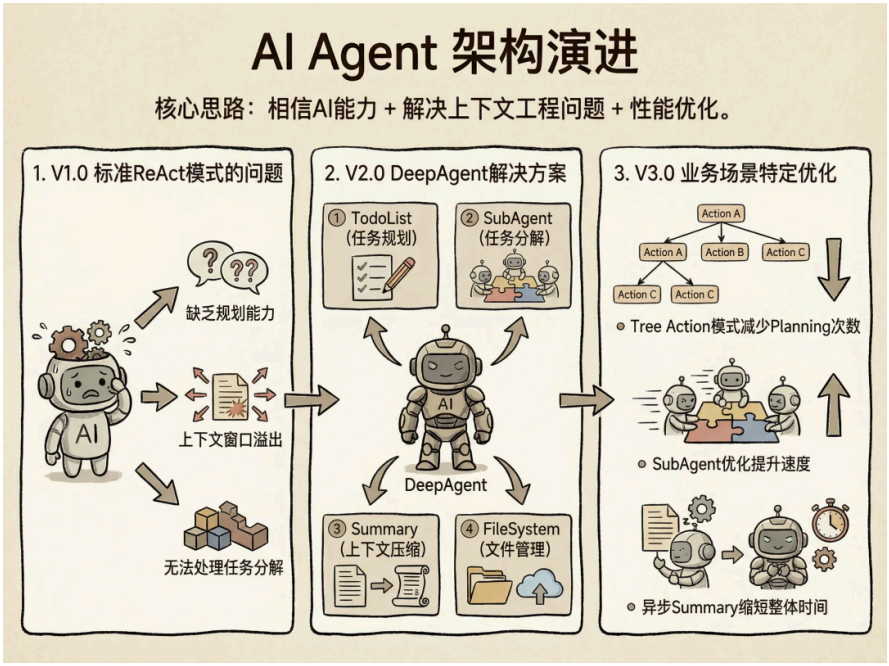
- 将完整的执行历史、取数过程、中间上下文信息传递给分析环节
- 支持基于全局思考过程的深度分析和跨模块综合分析
- 解决定制化workflow"分析数据来源单薄"的问题

- 核心设计思路：相信AI能力，解决上下文工程问题

设计理念

随着大模型基座能力的不断提升，理论上当上下文信息足够丰富、准确时，AI可以很好地完成大部分工作。因此，我们的核心思路是：**相信AI能力，将大量的工具调用、定制逻辑交给AI去自主完成**，充分发挥大模型的规划、分析和决策能力。同时，将工作重点转向解决AI在实际应用中面临的上下文工程问题，确保AI能够获得足够丰富、准确的上下文信息。

- 解决上下文工程问题



标准ReAct模式会遇到的问题：

理论上，我们只需要喂给大模型一堆工具让他自己去解决。但实际上，标准ReAct模式会遇到三大问题：

1. **缺乏规划能力**：代理无法提前规划任务步骤，无法跟踪任务进度，面对复杂任务时容易"迷失方向"
2. **上下文窗口溢出**：大量工具调用结果累积在上下文中，长对话历史占满token预算
3. **无法处理任务分解**：所有任务都在同一个上下文中执行，复杂任务会导致上下文混乱，无法并行执行独立任务

DeepAgent的解决方案

参考DeepAgent的设计思路，通过以下核心组件解决上述问题：

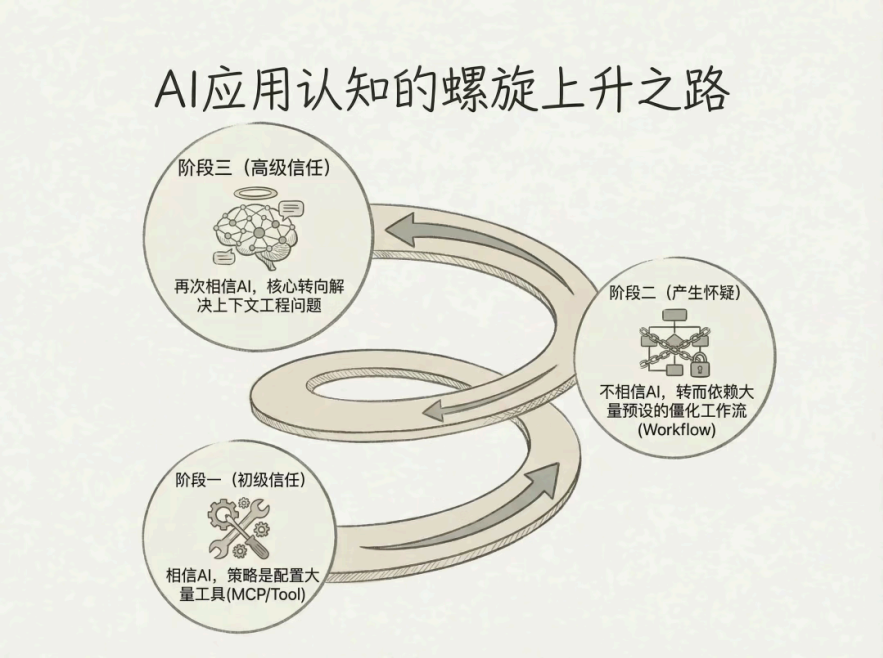
- TodoList：任务规划与进度跟踪，让Agent具备主动规划能力
- **SubAgent**：任务分解和上下文隔离，支持并行执行独立任务
- **Summary**：上下文压缩机制，自动监控并压缩历史消息，保留关键信息
- **FileSystem**：上下文管理机制，自动将大型工具结果保存到文件系统，不在上下文中保留，支持分页读取，避免一次性加载大文件，根据模型能力自动调整阈值

针对业务场景的特定优化

在DeepAgent基础上，针对业务场景做了大量速度优化和特定设计：

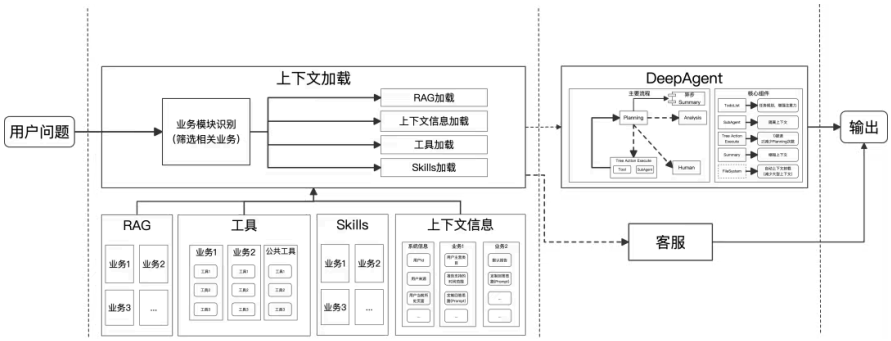
- **Tree Action模式**：按依赖关系执行Action Tree，减少Planning次数，提升执行效率
- **SubAgent优化**：简化SubAgent职责，聚焦核心任务执行，提升执行速度
- **异步Summary**：将Summary从串行改为异步执行，缩短整体响应时间

核心思路：相信AI能力 + 解决上下文工程问题 + 性能优化



03 DeepAgent模式技术架构详解

整体架构设计



架构概览

DeepAgent模式整体架构由两个核心模块组成：**加载上下文模块**和**DeepAgent模块**。这种设计将上下文准备与核心推理分离，既减轻了Planning阶段的压力，又确保了AI能够获得足够丰富、准确的上下文信息。

两个核心模块

1. **加载上下文模块**：负责在DeepAgent执行前，提前识别业务范围、加载相关资源，为后续的Planning和执行阶段做好准备。具体职责包括：
    - 业务模块识别：从单一业务模块升级为多业务模块识别，支持跨模块问题回答

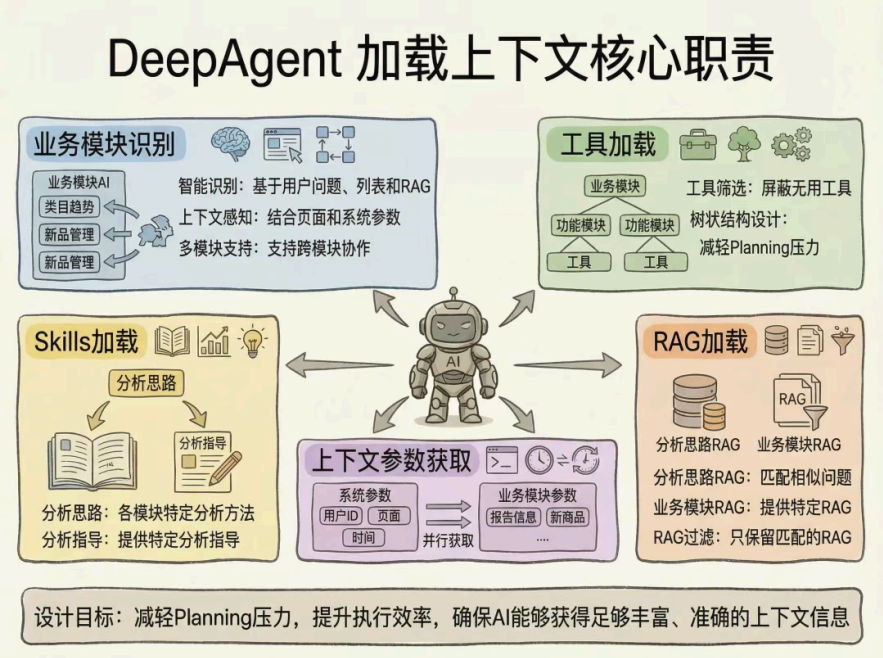
- 工具加载：根据业务模块识别结果，筛选并加载可能用到的工具，减轻Planning压力
- Skills加载：加载各个业务模块对应的特定分析思路和分析方法（Skills），为AI提供分析指导
- RAG加载：加载分析思路RAG和业务模块RAG，提供相似问题的分析思路和参考案例
- 上下文参数获取：提前获取业务模块所需的通用上下文参数，如用户的主营类目、洞察支持的时间范围等

2. **DeepAgent模块**：基于DeepAgent设计思路，包含Planning、Execute、Summary、Analysis等核心组件，负责任务规划、工具执行、上下文管理和最终分析。具体职责包括：
   - 任务规划：通过Planning组件进行任务规划和智能决策
   - 工具执行：通过Execute组件执行工具调用和SubAgent调用
   - 上下文压缩：通过Summary组件自动监控上下文窗口使用情况，当达到阈值时自动压缩历史消息，保留关键信息
   - 数据分析：通过Analysis组件进行深度分析，生成最终结论

设计理念

- **参考DeepAgent**：采用DeepAgent的核心组件（TodoList、SubAgent、Summary、FileSystem），解决标准ReAct模式的三大问题
- **业务特定优化**：针对业务场景做了大量速度优化和特定设计，如Tree Action模式、SubAgent优化、异步Summary等
- **上下文工程**：通过加载上下文模块，提前准备丰富的上下文信息，确保AI能够获得足够的信息进行决策

加载上下文模块



- 功能说明

加载上下文模块是DeepAgent模式的前置模块，负责在DeepAgent执行前，提前识别业务范围、加载相关资源，为后续的Planning和执行阶段做好准备。该模块的设计目标是减轻Planning压力，提升执行效率，确保AI能够获得足够丰富、准确的上下文信息。

核心功能

加载上下文模块主要包含以下五个核心功能：

- **业务模块识别**：根据用户问题识别涉及的业务模块，判断用户问题和哪些业务模块相关，为后续加载相关工具、Skills、RAG等提供基础。
- **加载工具**：根据业务模块识别结果，从工具库中筛选并加载相关的工具，避免将所有工具都加载到上下文中，减少上下文窗口占用，降低决策成本。
- **加载Skills**：加载业务模块特定的分析思路和最佳实践，提供业务定制逻辑（如当获取不到类目时使用主营类目，当获取多个价格带时使用GMV占比最高的价格带等），指导AI按照业务规范进行分析。
- **加载上下文参数**：提前获取系统参数和业务模块参数（如用户ID、主营类目等），为Planning阶段提供必要的业务背景信息。对于数据量较大的参数，采用智能截断策略，只加载前几条命中率最高的数据，其余数据通过公共接口供Planning按需调用。
- **加载RAG**：加载相似问题的历史案例和分析思路，为Planning提供参考示例和方向指导，确保分析结果符合业务定制的分析思路。

通过这五个核心功能，加载上下文模块能够为DeepAgent提供完整、准确、有针对性的上下文信息，确保Planning和执行阶段能够高效、准确地完成任务。

- 遇到的问题及解决方案

在定制化workflow架构下，我们遇到了以下问题，并通过加载上下文模块来解决：

问题1：工具过多导致上下文混乱和决策困难

- **问题描述**：通过评估，平均每个业务模块会有10–20个工具，那么随着业务模块的变多，会有大量工具。如果将大量的工具直接喂入给Planning，会导致上下文混乱，影响决策效率和准确性。例如，用户明明问的问题是和A业务相关的，但是喂入了大量的其他业务工具，会导致Planning决策起来困难，并且还有可能决策错误。同时，大量工具也会导致上下文窗口占用过多，成本急剧上升。

方案1：提前决策（当前采用）

- **方案描述**：通过提前判断用户问题的业务范围，缩小工具的选择范围。在DeepAgent执行前，通过业务模块识别机制，智能识别可能涉及的业务模块，然后根据业务模块识别结果，筛选并加载可能用到的工具，屏蔽不会用到的工具。
- **优点**：
  - 减少Planning需要处理的工具数量，降低上下文成本，同时提高决策的准确性和效率，避免无关工具干扰
  - 业务模块识别准确率高，效果稳定，具有良好的可观测性和可评测性，便于监控和优化
  - 提前划分了业务模块范围，可以尽量少的加载RAG、Skill、上下文信息，进一步降低上下文成本
- **缺点**：
  - 关键在于必须要保证业务模块识别的准确性，如果业务模块识别错误，那么后续全错了。业务模块识别是整个流程的关键环节，一旦识别错误，会导致工具选择范围过窄或选择错误的工具，进而影响整个任务的执行效果

方案2：Planning阶段自动加载（未采用）

- **方案描述**：让大模型在Planning阶段自动加载，给Planning赋予一个工具列表获取工具，这样Planning可以自己决策什么时候用哪些工具。
- **优点**：
  - Planning可以自主决定使用哪些工具，灵活性更高
  - 不需要提前进行业务模块识别，简化了前置流程
- **缺点**：
  - 关键在于必须要保证业效果不稳定，Planning可能选择错误的工具或工具选择范围过大
  - 速度变慢，Planning需要处理大量工具信息，影响决策效率
  - 上下文成本高，需要将大量工具信息传递给Planning
  - 决策困难，大量无关工具会干扰Planning的判断

综合考虑，我们采用了**方案1（提前决策）**，因为它能够有效减少工具数量，降低上下文成本和决策难度，同时保证执行速度和效果稳定性。

问题2：如何保证AI按照理想方式执行并兼顾业务定制分析思路

- **问题描述**：由于将大量的决策任务都交给了AI，那么如何保证AI能够按照我们理想的方式去做，同时如何兼顾业务的一些定制分析思路。如果没有明确的指引，AI可能按照自己的理解去规划任务，导致分析思路偏离业务预期，无法满足业务特定的分析需求。
- **解决方案**：通过RAG和Skills机制来解决这个问题，提前加载Skills和RAG资源，为AI提供业务分析思路指导。
  - **Skills的作用**：提供业务模块特定的分析思路和最佳实践，确保AI理解如何分析该业务模块的问题。Skills包含通用的分析思路和方法，为AI提供业务模块特定的分析指导，帮助AI更好地理解如何分析该业务模块的问题，确保分析方向符合业务预期。同时，Skills还包含一些业务定制逻辑，例如某些业务模块当获取不到类目时，可以用主营类目；当获取多个价格带时，使用GMV占比最高的价格带。这些业务定制逻辑确保AI在处理边界情况时能够按照业务规则执行，而不是依赖AI的自主判断。
  - **RAG的作用**：提供相似问题的历史案例和分析思路，为Planning提供方向指导。RAG通过检索相似问题的历史案例，提供具体的案例和参考，确保AI按照业务定制的分析思路进行规划，而不是依赖AI的自主发挥。
  - **组合效应**：Skills和RAG相互配合，Skills提供通用的分析思路和方法，RAG提供具体的案例和参考，共同确保AI按照业务定制的分析思路进行规划，保证AI能够按照理想的方式执行并兼顾业务定制分析思路。

问题3：上下文参数获取时机和策略

- **问题描述**：在实际分析过程中会依赖到很多系统参数、公共参数等，很多接口需要用到，涉及到用户id、主营类目等。那么在什么时机去加载这些参数呢？如果将这些参数获取都放在Planning阶段，会导致Planning需要多次调用接口获取参数，影响执行效率。
- **解决方案**：采用前置的分析思路，将这些信息提前准备好，以便Planning直接去使用。但并非所有上下文信息都适用，有些上下文信息返回的数据过多，所以只会进行截断，将这些内容抽象出一些公共接口，以便后续Planning阶段再调用。例如，商家的新品列表通常涉及到分页，因此只会提前加载出前5条（命中率最高的），剩下的由Planning在后续去决定是否需要使用。这样既减少了Planning阶段的接口调用次数，提升了执行效率，又避免了上下文成本过高的问题。

- 详细设计

业务模块识别

区别于定制化workflow的单一业务模块定位，DeepAgent模式支持多业务模块识别，从之前的单一业务模块升级为多个业务模块，支持跨模块问题回答。

- **智能识别**：基于用户问题、业务模块列表和RAG，智能识别可能涉及的业务模块

- **上下文感知**：结合用户当前所处的页面和系统参数，更准确地识别业务模块
- **多模块支持**：如果用户问题涉及多个业务模块，可以返回多个业务模块名称，支持跨模块协作

示例说明：

- **单模块问题**：用户问题"2025年5月女装连衣裙类目下销售额为多少？"只需要使用类目趋势报告模块，属于单模块问题。
- **多模块问题**：用户问题"我想研发美容护肤类的新品，请帮我分析一下我适合研发哪类新品（选10个竞争强度最低的叶子类目），并帮我分析一下我可能对标的竞品（这些机会类目对应的Top20竞品）有什么特点？"需要同时使用类目趋势报告模块（获取类目数据和竞争强度分析）和新品管理模块（获取竞品数据），属于多模块问题，需要跨模块协作。

Prompt设计要点

业务模块识别的Prompt设计需要明确以下要点：

- **角色定位**：业务模块识别专家，负责根据用户问题和当前上下文，准确识别用户意图对应的业务模块
- **核心目标**：精准识别、意图理解、上下文感知、多模块支持
- **识别原则**：
  - **精准匹配**：优先选择与用户问题直接相关的业务模块，将用户问题与业务模块描述进行匹配
  - **上下文参考**：可以参考用户当前所处页面，但应以用户问题的实际意图为准
  - **多模块支持**：如果用户问题涉及多个业务模块，可以返回多个业务模块名称
  - **避免误判**：如果用户问题与任何业务模块都不匹配，返回空列表

工具加载

根据业务模块识别结果，筛选并加载可能用到的工具，减轻Planning压力。

工具筛选机制

- **按业务模块筛选**：根据业务模块识别结果，筛选出可能用到的工具，屏蔽不会用到的工具，减轻Planning压力。例如，如果识别到"新品管理"业务模块，则只加载新品管理模块下的工具，不会加载"AI行业趋势报告"或"细分市场洞察"模块的工具。
- **动态加载**：根据用户问题的业务范围，动态加载相关工具，确保Planning只看到与当前问题相关的工具，避免工具过多导致上下文混乱。

树状结构设计

工具箱采用三层树状结构设计，便于管理和筛选：

- **第一层：业务模块**：如"新品管理"、"AI行业趋势报告"、"细分市场洞察"、"站内趋势"等，每个业务模块对应一个独立的业务场景

- **第二层：功能模块**：每个业务模块下包含多个功能模块，如新品管理模块包含"公共"、"人群分析"、"沟通策略"、"新品情报"、"竞争分析"、"经营分析"、"深度洞察"等功能模块
- **第三层：工具**：每个功能模块下包含具体的工具，如"人群分析"功能模块包含"queryNewItemCrowdInfo"（获取类目下的人群信息）等工具

公共工具设计

除了各业务模块的专用工具外，系统还设计了公共工具模块，包含跨业务模块的通用工具：

- **公共工具的作用**：提供各业务模块都可能用到的通用工具，避免在每个业务模块中重复定义。例如，类目识别工具（publicCategoryRecognize）是一个典型的公共工具，无论是新品管理、AI行业趋势报告还是其他业务模块，都可能需要识别用户问题中提到的类目。
- **为什么需要公共工具**：
  - **避免重复定义**：如果每个业务模块都定义自己的类目识别工具，会导致工具定义重复，增加维护成本
  - **提高复用性**：公共工具可以在多个业务模块中复用，提高工具的利用率
  - **统一行为**：公共工具确保跨业务模块的一致性行为，例如类目识别逻辑在所有业务模块中保持一致
  - **降低上下文成本**：公共工具只需要加载一次，而不是在每个业务模块中都加载，降低上下文成本

公共工具会在所有业务模块识别完成后统一加载，确保Planning可以使用这些通用工具。

Skills加载

加载各个业务模块对应的特定分析思路和分析方法（Skills），为AI提供分析指导。

Skills的核心内容

Skills包含业务模块特定的分析思路、最佳实践和业务定制逻辑，确保AI能够按照业务预期进行分析。以"新品管理"模块为例，Skills包含以下内容：

- **默认选择逻辑**：为AI提供明确的默认选择规则，当用户问题不够明确时，AI可以按照这些规则自动选择，而不是询问用户。例如：
  - **新品的默认选择逻辑**：如果用户的问题中并没有明确指出其想要查看的新品范围，那么可以遵循以下规则：1)用户提到了"我的新品"那么就优先新品列表中的第一个品；2)如果什么都没有提到则看用户是否正处于某份报告/商品的页面；3)如果都没有则默认第一个商品/报告。
  - **价格带的默认选择逻辑**：如果用户明确提到了某个价格带、所有价格带、或者对比不同的价格带，则按照用户的要求去做；如果用户没有明确表明价格带，则默认选择GMV占比最高的价格带，并且在思考（thought）中告知用户已为它默认选择GMV占比最高的价格带。

- **业务定制逻辑**：包含业务模块特定的处理规则，确保AI在处理边界情况时能够按照业务规则执行。例如：
  - 某些业务模块当获取不到类目时，可以用主营类目
  - 当获取多个价格带时，使用GMV占比最高的价格带
  - 这些业务定制逻辑确保AI在处理边界情况时能够按照业务规则执行，而不是依赖AI的自主判断

Skills的作用

- **分析思路指导**：为AI提供业务模块特定的分析思路和最佳实践，帮助AI理解如何分析该业务模块的问题，确保分析方向符合业务预期
- **自动化决策**：通过默认选择逻辑，让AI能够自动处理模糊问题，减少与用户的交互，提升用户体验
- **业务规则约束**：通过业务定制逻辑，确保AI在处理边界情况时能够按照业务规则执行，保证分析结果的准确性和一致性

RAG加载

加载分析思路RAG和业务模块RAG，提供相似问题的分析思路和参考案例。

- **分析思路RAG**：查询分析思路RAG库，匹配到相似问题的分析思路
- **业务模块RAG**：获取业务模块对应的特殊分析RAG，提供业务模块特定的分析指导
- **RAG过滤**：根据业务模块识别结果，过滤RAG，只保留业务模块匹配的RAG

详细设计见上文（RAG设计）

Skills和RAG的重要性

在DeepAgent模式下，我们将大量的工具交给AI去做，让AI负责Planning。为了保证AI能够按照理想的、具有业务分析思路的方式进行分析，就必须要通过大量的Skills和RAG去指引。

- **Skills的作用**：为AI提供业务模块特定的分析思路和最佳实践，帮助AI理解如何分析该业务模块的问题，确保分析方向符合业务预期
- **RAG的作用**：通过相似问题的历史案例和分析思路，为Planning提供方向指导，确保AI按照业务定制的分析思路进行规划，而不是依赖AI的自主发挥
- **组合效应**：Skills和RAG相互配合，Skills提供通用的分析思路和方法，RAG提供具体的案例和参考，共同确保AI的分析路径与业务预期保持一致

上下文参数获取

提前获取业务模块所需的通用上下文参数，为后续的工具调用和分析提供基础信息。这样可以减少Planning阶段的接口调用次数，提升执行效率，同时避免上下文成本过高的问题。

系统参数

系统参数是所有业务模块都可能用到的通用参数，包括：

- **用户Id（userId）**：用于标识当前用户，几乎所有工具调用都需要此参数
- **用户来源（userSource）**：标识用户的来源渠道，用于数据权限控制
- **用户当前所处页面**：用户当前访问的页面信息，可以作为上下文参考，帮助更准确地理解用户意图
- **用户的主营类目**：用户主要经营的类目，当用户问题中没有明确指定类目时，可以作为默认类目使用
- **用户的提问时间**：用于时间相关的分析和数据查询

业务模块参数

根据识别到的业务模块，并行获取各个业务模块所需的通用上下文参数。不同业务模块需要不同的上下文参数：

- **类目趋势报告模块**：
  - 当前用户的主营类目：当用户问题中没有明确指定类目时，使用主营类目作为默认值
  - 报告支持的时间范围：了解报告支持哪些时间范围，避免查询不支持的时间
  - 定制回答思路Prompt：业务模块特定的回答思路和格式要求
- **新品管理模块**：
  - 用户最近的新品上新报告列表：如果用户问题中提到"我的新品"或"我的报告"，可以从列表中快速定位
  - 用户最近的Top10新品：当用户问题中没有明确指定新品时，可以作为默认选择
  - 从问题中识别到的类目：提前识别用户问题中提到的类目，避免在Planning阶段重复识别
- **细分市场洞察模块**：
  - 相关的细分市场报告信息：用户创建的细分市场报告列表，当用户问题中提到"在xxx报告中"时，可以快速定位

参数获取策略

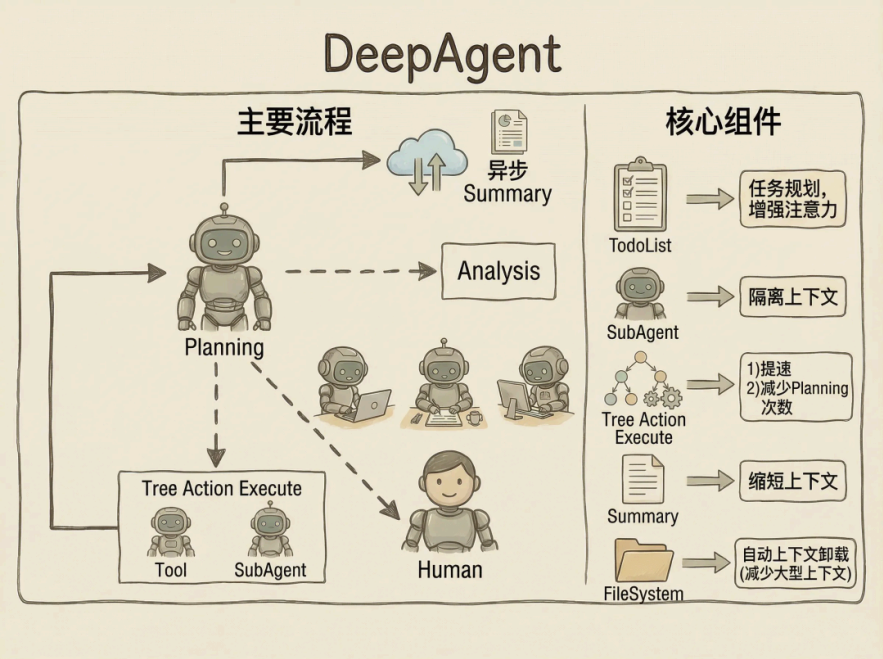
- **并行获取**：前置并行获取各个业务模块所需的通用上下文参数，提升效率。如果用户问题涉及多个业务模块，可以同时并行获取各个模块的参数，而不是串行获取，大幅缩短整体响应时间。
- **智能截断**：对于返回数据过多的上下文信息，采用截断策略，只加载命中率最高的部分数据，避免上下文成本过高。例如，商家的新品列表通常涉及到分页，因此只会提前加载出前5条（命中率最高的），剩下的由Planning在后续去决定是否需要使用。这样既减少了上下文成本，又保证了常用数据的可用性。

参数使用示例

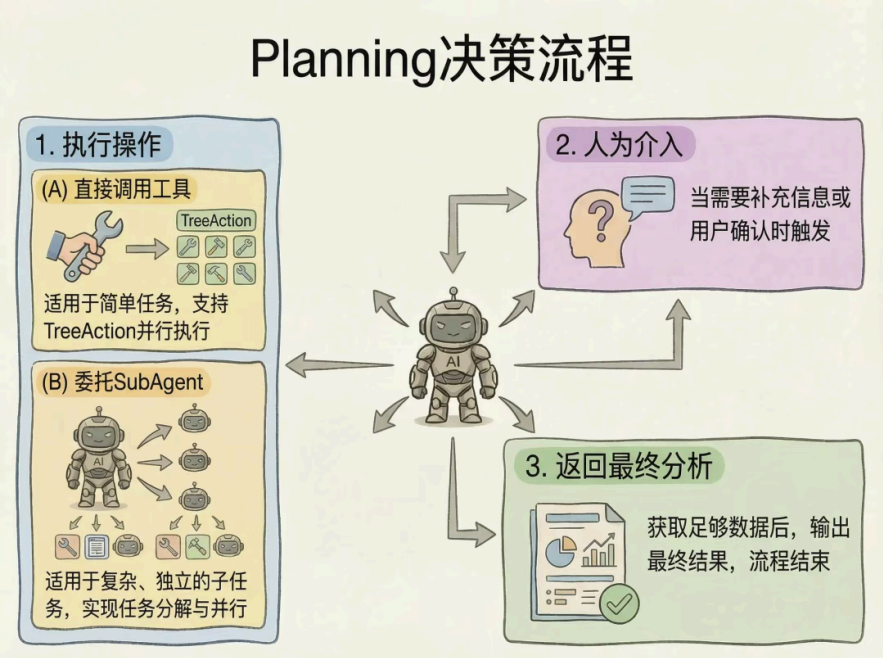
以"分析我的新品在类目市场中的表现"为例：

1. **系统参数获取**：获取用户Id、用户来源、用户当前所处页面、用户的主营类目等
2. **业务模块参数获取**：
    - 新品管理模块：获取用户最近的新品列表（前5条）、用户最近的新品上新报告列表
    - 类目趋势报告模块：获取用户的主营类目、报告支持的时间范围
3. **Planning阶段使用**：Planning可以直接使用这些预加载的参数，无需再次调用接口获取，例如可以直接使用新品列表中的第一个新品，或者使用主营类目作为默认类目。

DeepAgent核心模块



- Planning（规划器）



功能说明

Planning是DeepAgent模式的核心组件，负责在每一轮按照ReAct（Thought → Action → Observation）模式决定"下一步做什么"。Planning不直接执行工具，而是产生标准化的动作意图（Action intent），可以选择直接调用工具、委托给子代理、请求人为介入补充信息，或返回最终答案。

Planning的核心能力包括：

- **任务规划**：能够将复杂任务分解为清晰的执行计划（TodoList），并动态跟踪和更新任务状态
- **工具调用**：可以直接调用工具（取数工具、参数识别工具）来执行简单任务
- **数据分析**：能够对获取的数据进行深度分析，得出结论和分析思路，这是Planning的核心分析能力
- **子代理委托**：可以将复杂、多步骤的独立任务委托给子代理（SubAgent）执行，实现上下文隔离和并行处理
- **并行执行**：能够识别独立任务并并行执行，最大化执行效率
- **智能决策**：基于用户输入、会话上下文、执行历史和工具返回结果，智能决定下一步行动

功能说明

遇到的问题及解决方案

问题1：如何设计上下文信息

- **问题描述**：为了保证充足的上下文信息，Planning需要哪些上下文信息，上下文包含哪些内容。如果上下文信息不足，AI可能无法做出正确的决策；如果上下文信息过多，可能导致上下文混乱和成本过高。

- **解决方案**：通过加载上下文模块提前准备上下文信息，Planning的上下文信息包含以下8个部分，每一部分都有其特定的作用和引入原因：

  a. **用户原始问题**：用户提出的原始问题，是Planning进行任务规划和决策的基础。Planning需要基于用户问题理解用户意图，制定执行计划，决定下一步动作。

  b. **上下文参数信息**：包括系统参数（用户Id、用户来源、用户当前所处页面、用户的主营类目、用户的提问时间等）和业务模块参数（如新品列表、报告列表等）。这些参数是工具调用和分析的基础信息，提前加载可以减少Planning阶段的接口调用次数，提升执行效率。

  c. **业务模块对应的特定分析思路（Skills）**：针对每个业务模块的定制分析思路，包含各个业务模块的定制逻辑和工具使用组合逻辑。Skills为不同业务模块提供了针对性的分析思路和最佳实践，确保AI能够按照业务预期进行分析，帮助Planning按照业务规范进行规划，而不是依赖AI的自主发挥。

  d. **工具列表**：根据业务模块识别结果筛选出的可能用到的工具。减少Planning需要处理的工具数量，降低上下文成本和决策难度，Planning可以根据工具列表选择合适的工具来执行任务，避免在大量无关工具中做选择。

  e. **执行历史**：Planning和工具执行的历史记录。帮助Planning了解已经执行过的操作和获取到的数据，避免重复执行，并基于历史结果做出下一步决策，判断任务进度，决定是否需要继续执行或调整执行计划。

  f. **上一轮的执行计划（TodoList）**：上一轮Planning生成的执行计划。帮助Planning了解当前任务的整体规划，跟踪任务进度，动态更新执行计划。Planning可以基于TodoList了解任务分解情况，更新任务状态，识别已完成和待完成的任务。

  g. **参考的分析思路（RAG）**：通过用户问题召回到的相似问题的分析方法和思路。为Planning提供历史成功案例的分析思路，确保AI按照业务定制的分析思路进行规划。Planning可以参考相似问题的分析方法，理解如何分析类似问题，选择合适的工具和数据字段。

  h. **对话历史**：用户与系统的历史对话记录。帮助Planning理解用户问题的上下文和背景，更准确地生成执行计划和判断下一步动作。Planning可以结合对话历史理解用户意图的演变，处理多轮对话中的上下文依赖关系。

问题2：如何确保每一轮Planning不会偏离主线

- **问题描述**：因为会进行一轮一轮的Planning，如何确保每一轮Planning的时候不会偏离主线，是有一个中心在那里的。在多轮Planning过程中，如果没有一个明确的中心来约束和引导，AI可能会在每一轮中做出不同的决策，导致任务执行偏离原始目标，无法完成用户的核心需求。

- **解决方案**：通过TodoList机制来解决这个问题。TodoList作为Planning的中心和主线，在任务开始时生成完整的执行计划，明确任务分解和依赖关系。在每一轮Planning中，TodoList都会被传递给Planning，作为当前任务的整体规划参考。Planning可以基于TodoList了解任务分解情况，更新任务状态，识别已完成和待完成的任务，确保每一轮的决策都围绕TodoList这个中心展开，不会偏离主线。同时，TodoList支持动态更新，可以根据执行情况调整任务计划，但始终保持在主线的约束范围内。

示例说明：以用户问题"分析我的新品在类目市场中的表现"为例：

- **第一轮Planning**：生成TodoList，包含任务1（获取新品列表）、任务2（获取类目市场数据）、任务3（分析新品在类目市场中的表现）。TodoList明确了任务分解和依赖关系。
- **第二轮Planning**：执行完任务1和任务2后，Planning会看到TodoList中任务1和任务2已完成，任务3待完成。此时Planning会围绕任务3展开，继续分析新品在类目市场中的表现，而不会偏离到其他无关任务（如分析竞品、分析价格带等）。
- **第三轮Planning**：如果需要进一步获取数据来完成分析，Planning会基于TodoList中的任务3，继续执行相关操作，始终围绕"分析新品在类目市场中的表现"这个主线，不会偏离。

通过TodoList这个中心，每一轮Planning都能清楚地知道当前任务的整体规划和进度，确保决策始终围绕主线展开。

问题3：执行动作设计

- **问题描述**：Planning需要决定"下一步做什么"，那么应该设计哪些执行动作来覆盖所有可能的场景。如果执行动作设计不合理，可能导致Planning无法准确表达意图，或者执行动作过于复杂难以理解和使用。
- **解决方案**：设计三种执行动作来覆盖所有可能的场景：call_actions（执行操作）、call_human（人为介入）、final_analysis（返回最终分析）。这三种动作的设计原因如下：
  - **call_actions（执行操作）**：这是Planning最常用的执行动作，用于执行具体的操作，包括直接调用工具和委托给SubAgent执行。设计这个动作的原因是需要支持Planning主动执行任务，获取数据或执行分析。call_actions可以包含多个操作，支持并行和串并行混合执行，满足复杂任务的需求。
  - **call_human（人为介入）**：当Planning需要额外的信息或确认才能继续执行任务时使用。设计这个动作的原因是在某些场景下，AI无法自动获取必要信息（如用户偏好、业务规则确认等），需要人工介入补充信息。但需要注意，如果业务模块对应的特定分析思路中提供了默认选项，必须遵循这些默认选项，直接使用默认值继续执行，不应使用call_human。
  - **final_analysis（返回最终分析）**：当已经获取足够的数据并完成分析后使用。设计这个动作的原因是需要一个明确的结束标志，告诉系统Planning阶段已经完成，可以进入Analysis阶段进行最终分析和输出。一旦进入返回最终分析，流程将不会再回到Planning继续执行，会直接交由Analysis进行分析并最终输出给用户。

通过这三种执行动作，Planning可以覆盖所有可能的场景：继续执行任务（call_actions）、需要人工介入（call_human）、完成任务（final_analysis），形成一个完整的执行动作体系。

问题4：错误处理机制的设计

- **问题描述**：在执行过程中，Planning会遇到各种异常情况，如数据查询为空、工具执行错误、参数识别错误、SubAgent执行失败等。如果没有完善的错误处理机制，可能导致Planning陷入死循环、重复执行失败的操作，或者无法优雅地处理错误，影响用户体验。

- **解决方案**：设计完善的错误处理机制，针对不同类型的错误采用不同的处理策略
  - **数据查询为空**：不要硬分析，直接告知用户查询不到数据，可以建议用户调整查询条件，返回最终分析并说明查询结果为空。这样避免AI基于空数据做出错误的分析结论。
  - **工具执行错误：**
    - **禁止原参数重试**：如果工具执行失败，绝对不要使用相同的参数和工具再次尝试，避免陷入死循环。
    - **处理策略**：如果错误是永久性的（如参数错误、权限不足、资源不存在），直接返回最终分析并说明错误原因；如果错误可能是参数或工具问题，可以尝试使用替代方案，如果替代方案也失败，则返回最终答案。
  - **参数识别错误：**使用上下文参数信息进行验证，如果参数不完整，先调用参数识别工具，如果参数识别工具也失败，返回最终答案说明无法识别必需参数。
  - **SubAgent执行失败**：如果SubAgent返回错误，评估是否可以重试或使用替代方案，如果无法继续，返回最终答案说明子代理执行失败的原因。

通过完善的错误处理机制，Planning能够优雅地处理各种异常情况，避免陷入死循环，并能够向用户清晰地说明错误原因，提升用户体验。

- Tree Action（混合执行模式）

功能说明

Tree Action是一种混合执行模式，支持在单轮Planning中进行并行以及串并行混合执行，所有操作都在单次Execute中完成。Tree Action模式既支持无依赖关系的并行执行，也支持有依赖关系的串行执行，实现了真正的混合执行。

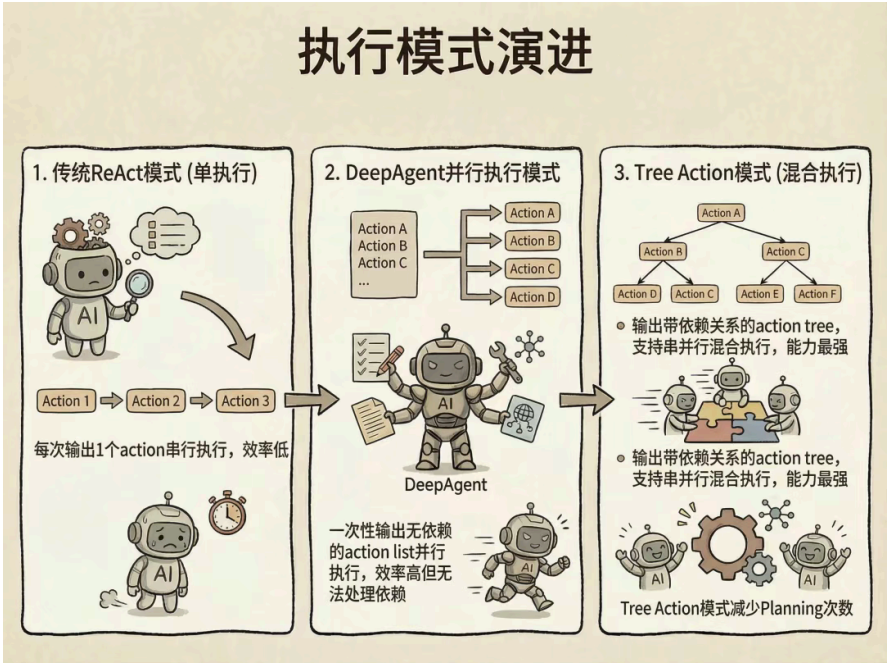
执行模式演进

从传统ReAct到DeepAgent，再到Tree Action模式，执行模式经历了三个阶段的演进：

- **传统ReAct模式（串行执行）**：
  - 每次Planning只输出一个action，串行执行
  - 执行效率低，无法充分利用并行能力
  - 无法处理工具之间存在依赖关系的情况，需要多次Planning往返
- **DeepAgent并行执行模式**：
  - 每次Planning输出一个action list，这些Action是不具备依赖关系的，支持并行执行
  - 大幅提升执行效率，缩短整体响应时间
  - 但仍无法处理工具之间存在依赖关系的情况
- **Tree Action模式（混合执行）**：
  - 每次Planning输出一个具有依赖关系的Action Tree，按照各个Action之间的依赖关系顺序执行

- 支持在单轮中进行并行以及串并行混合执行，所有操作都在单次Execute中完成
- 既支持无依赖关系的并行执行，也支持有依赖关系的串行执行，实现真正的混合执行

Tree Action模式的核心价值在于解决了工具之间存在依赖关系时的执行效率问题。通过依赖关系标记，Planning可以在单次输出中生成完整的Action Tree，系统自动按依赖顺序执行（先执行第一层无依赖的Actions并行执行，再执行第二层依赖第一层的Actions，以此类推），大幅提升执行效率。



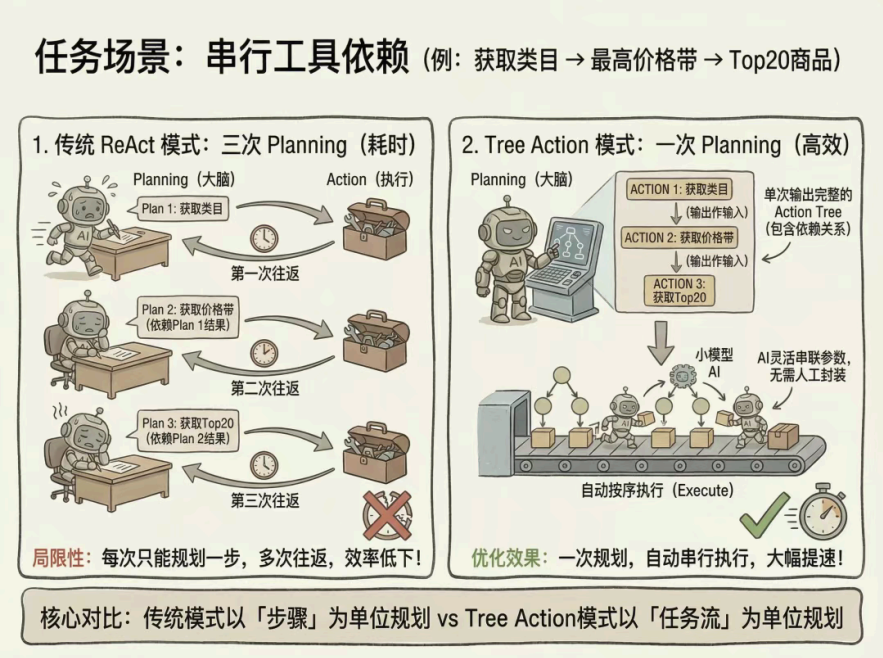
功能说明

问题1：工具依赖关系导致执行效率低

- **问题描述**：在实际执行过程中，经常会遇到工具之间存在依赖关系的情况。例如，以"获取类目、获取最高的价格带、获取最高价格带下的Top20商品"这个需求为例，这三个操作本质上是串行的：工具2（获取价格带）的输入依赖工具1（获取类目）的输出，工具3（获取Top20商品）的输入依赖工具2的输出。

在传统ReAct模式下，每次Planning只能输出一个action，因此需要三次Planning往返（第一次Planning输出获取类目，执行完成后第二次Planning输出获取价格带，执行完成后第三次Planning输出获取Top20商品），导致时间大大增加，执行效率低。

在DeepAgent并行执行模式下，虽然支持并行执行，但只能处理无依赖关系的Actions，无法处理工具之间存在依赖关系的情况。



- **传统解决方案的局限性**：

  面对工具依赖关系，传统工程手段存在以下局限性：

  - **封装成大Tool**：将三个串联的工具封装成一个大的Tool，但工具的组合关系是不可枚举的，会增加很多人力成本去组合这些工具
  - **定制SubAgent**：定制一个SubAgent专门负责解决这个问题，但同样面临工具组合关系不可枚举的问题，且需要为每种组合关系定制SubAgent，成本高昂

- **解决方案：Tree Action模式**

  Tree Action模式通过AI的灵活性来解决这个问题：

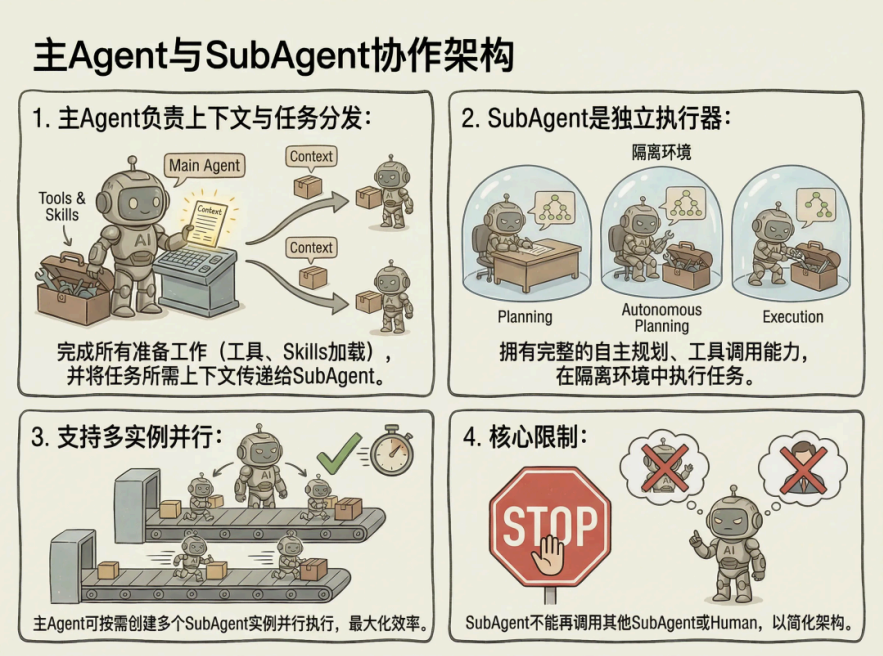
  - **依赖关系标记**：Planning可以在单次输出中生成完整的Action Tree，通过依赖关系标记表达工具之间的依赖关系
  - **自动按序执行**：系统自动按依赖顺序执行（先执行第一层无依赖的Actions并行执行，再执行第二层依赖第一层的Actions，以此类推），所有操作都在单次Execute中完成
  - **AI串联工具**：因为工具2的输入依赖工具1的输出，使用传统的工程手段很难去串联前一个工具的输出和后一个工具的输入。而各个工具之间的串联通常比较简单，需要的上下文信息不多，因此可以通过调用小模型去负责这里的工具串联和参数传递。小模型具备理解依赖关系和参数传递的能力，可以灵活地串联工具，同时具备很高的灵活性和低延时

- **优化效果**

  - **减少Planning次数**：避免多次Planning往返，从三次Planning减少到一次Planning，大幅提升执行效率
  - **灵活性**：相比将工具组合封装成一个大Tool或定制SubAgent，Tree Action模式更加灵活，无需枚举所有工具组合关系，AI可以根据实际需求动态组合工具

测试结论：经评估，在我们的测试集上，Tree Action模式将Planning次数从3.20次优化到2.44次，显著提升了执行效率。

SubAgent（子代理）



功能说明

SubAgent是DeepAgent实现任务分解和上下文隔离的核心组件，参考DeepAgent的设计。SubAgent的核心机制包括：

- **上下文隔离**：子代理拥有独立的上下文窗口，不污染主代理的上下文
- **任务分解**：将复杂多步骤任务分解为独立的子任务
- **并行执行**：支持同时启动多个子代理并行工作，最大化执行效率
- **简洁结果**：子代理返回简洁的执行结果，而非完整的执行历史

SubAgent架构

SubAgent和主Agent的结构一模一样，其核心为DeepAgent模块，包含Planning、Execute、Summary、Analysis等核心组件。SubAgent与主Agent的主要区别在于：

- **不需要上下文加载**：SubAgent不需要进行上下文加载（业务模块识别、工具加载、Skills加载、RAG加载、上下文参数获取等），这些工作由主Agent完成
- **上下文由主Agent传递**：SubAgent所需的相关上下文信息（包括已识别的参数、业务模块信息、工具信息等）都由主Agent通过subAgentDescription明确传递给SubAgent
- **独立执行**：SubAgent在独立的上下文窗口中执行，拥有完整的DeepAgent能力，可以自主进行Planning、工具调用、数据分析等操作
- **限制**：SubAgent不能再调用SubAgent和Human，避免嵌套过深和人为介入的复杂性

SubAgent实例化

SubAgent在执行时类似于实例的概念，可以创建多个SubAgent实例：

- **实例化机制**：主Agent可以根据任务需求创建多个SubAgent实例，每个实例在独立的上下文窗口中执行
- **实例数量由主Agent决定**：主Agent根据执行计划中的任务需求，决定创建几个SubAgent实例。例如，如果需要"分别获取三个类目的数据"，主Agent可以创建3个SubAgent实例，每个实例负责获取一个类目的数据
- **并行执行**：多个SubAgent实例可以并行执行，互不干扰，最大化执行效率

SubAgent类型

DeepAgent模式支持两种SubAgent类型：

- **通用SubAgent**（已引入）：
  - **核心内容**：等同于主DeepAgent，其区别在于不能再调用SubAgent和Human
  - **主要作用**：隔离上下文，实现任务分解和并行执行
  - **使用场景**：适用于需要上下文隔离、任务分解或并行执行的场景
  - **优势**：具备主DeepAgent的所有能力（除调用SubAgent和Human外），可以灵活处理各种子任务
- **定制SubAgent**（未引入）：
  - **核心内容**：定制Agent，具有特定的分析目的（例如归因分析Agent等）
  - **主要作用**：解决特定场景下的问题
  - **未引入原因**：定制SubAgent本质上是通过枚举的方式专门去解某一类问题，目前还没有这种场景需求。通用SubAgent已经足够灵活，可以通过Prompt和上下文信息引导AI完成各种任务，无需为特定场景定制专门的SubAgent

遇到的问题及解决方案

问题1：SubAgent执行效率优化

- **问题描述**：SubAgent处理的任务多半为子任务，不会过于复杂，如果使用完整的DeepAgent能力（包括完整的TodoList规划和详细的分析过程），会导致执行效率低，增加不必要的开销。
- **解决方案**：针对业务场景，对SubAgent进行了优化，简化其职责以提升执行速度：
  - **简化Planning职责**：删除TodoList，减少Planning节点的输出内容。SubAgent处理的任务多半为子任务，不会过于复杂，不需要完整的TodoList规划
  - **简化Analysis职责**：只输出结论，不需要过程和建议。SubAgent专注于执行子任务并返回结果，分析工作由主Agent完成

优化效果：通过简化SubAgent的职责，SubAgent的平均执行时间从103秒优化到20秒，显著提升了执行效率。

问题2：SubAgent上下文设计

- **问题描述**：SubAgent需要哪些上下文信息，如何设计SubAgent的上下文，确保SubAgent能够正确执行任务。如果上下文信息不足，SubAgent可能无法正确理解任务或获取必要信息；如果上下文信息过多，可能导致上下文混乱和成本过高。
- **解决方案**：SubAgent的上下文完全由主Agent决定，主Agent会准备好SubAgent需要的上下文。SubAgent不需要进行上下文加载（业务模块识别、工具加载、Skills加载、RAG加载、上下文参数获取等），这些工作由主Agent完成。主Agent通过subAgentDescription明确传递给SubAgent所需的上下文信息，包括已识别的参数、业务模块信息、工具信息等，SubAgent可以直接使用这些信息，无需重新识别。

示例说明：以主Agent给SubAgent的任务描述为例：

主Agent通过subAgentDescription传递给SubAgent的上下文信息包括：

- **任务描述**：分析'质价比/平替'类需求的趋势特征。从已获取的趋势词列表中，筛选与'性价比/平替/高保湿/基础护理'相关的趋势词（如'绵羊油凡士林锁水修护护手霜'、'冬季高保湿修护面霜'、'轻量环保便携面霜'等），获取这些趋势词的价格带分布数据和人群画像数据。
- **工具信息**：需要调用getTrendMarketPriceSeg和getTrendMarketCrowdDetail工具
- **已识别的参数**：类目ID为1801
- **任务要求**：请选取3–5个代表性趋势词进行分析，返回各趋势词的价格带销售额占比分布和人群画像特征（购买力等级、城市等级、年龄段等维度的占比）

SubAgent接收到这些上下文信息后，可以直接使用类目ID、工具列表等信息，无需重新识别类目或加载工具，专注于执行分析任务。主Agent已经完成了所有的上下文准备工作，SubAgent只需要按照任务描述执行即可。

通过这种上下文设计，SubAgent可以快速启动并执行任务，无需重复进行上下文加载，提升了执行效率。

问题3：并行设计

- **问题描述**：在实际执行过程中，有很多任务是分别下钻然后获取到某个东西，这些任务之间相互独立，可以并行执行。如果串行执行这些任务，会导致执行效率低，时间成本高。例如，需要分别获取多个类目的数据、分别分析多个趋势词的特征等。
- **解决方案**：通过SubAgent并行处理机制来解决这个问题。对于相互独立的子任务，主Agent可以创建多个SubAgent实例并行执行，每个SubAgent负责一个子任务的下钻和获取。主Agent只关心最终结果，不需要感知中间过程，可以同时启动多个SubAgent实例并行工作，最大化执行效率。

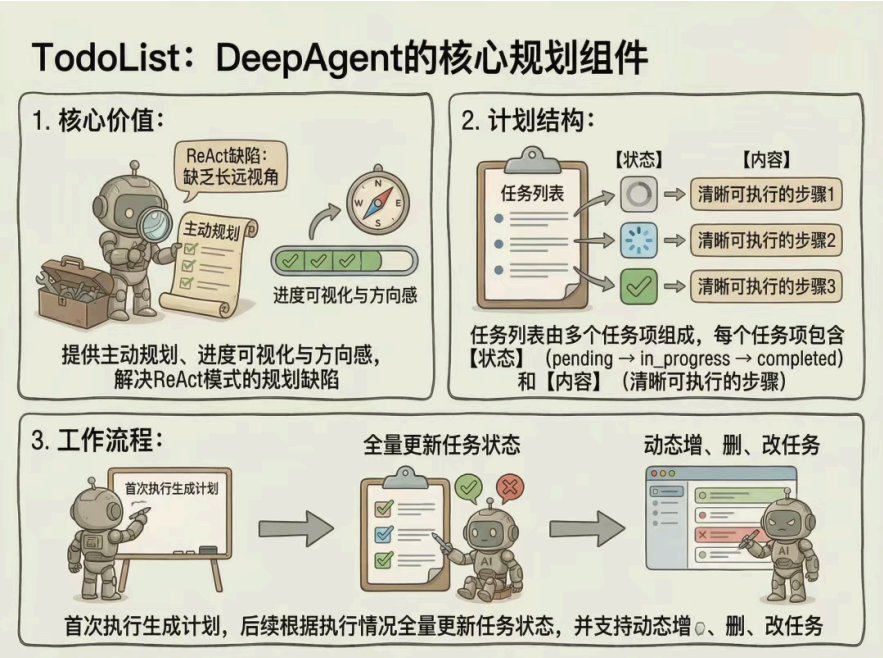
示例说明：以"获取每个类目最有潜力价格带下的典型商品"为例：

- **任务特点**：需要分别获取类目A、类目B、类目C的最有潜力价格带下的典型商品，每个类目都需要多步下钻：
  - 第一步：获取类目的价格带数据
  - 第二步：识别最有潜力的价格带
  - 第三步：获取该价格带下的典型商品

    这三个类目的任务相互独立，可以并行执行
- **并行执行**：主Agent创建3个SubAgent实例，每个实例负责一个类目的完整下钻链路：
  - SubAgent1：获取类目A的价格带数据 → 识别最有潜力价格带 → 获取典型商品
  - SubAgent2：获取类目B的价格带数据 → 识别最有潜力价格带 → 获取典型商品
  - SubAgent3：获取类目C的价格带数据 → 识别最有潜力价格带 → 获取典型商品
- **只关心结果**：主Agent只关心每个SubAgent返回的最终结果（典型商品列表），不需要感知每个SubAgent的中间执行过程（如获取价格带数据、识别最有潜力价格带等中间步骤），可以并行启动多个SubAgent实例，每个SubAgent独立完成自己的下钻链路，大幅缩短整体响应时间

通过SubAgent并行设计，系统可以充分利用并行能力，同时处理多个独立的下钻任务，显著提升执行效率。

TodoList（任务列表）



功能说明

TodoList是DeepAgent的核心规划组件，为代理提供任务规划与进度跟踪能力。允许代理在执行复杂任务前创建任务列表，并在执行过程中动态更新任务状态。TodoList是Planning的核心

规划能力，参考DeepAgent的设计，用于将复杂任务分解为清晰的执行步骤，并实时跟踪任务进度。

核心价值

- **主动规划能力**：让代理具备主动规划能力，而不是被动响应。代理可以在执行复杂任务前创建任务列表，提前规划执行步骤
- **进度可视化**：提供清晰的进度可视化和状态跟踪，实时反映每个步骤的执行状态，确保任务方向清晰
- **保持方向感**：帮助代理在面对复杂多步骤任务时保持方向感，不会在执行过程中迷失方向
- **任务依赖分析**：支持任务依赖分析和智能执行顺序决策，通过status的顺序来体现任务之间的依赖关系

执行计划的结构

执行计划是一个数组，每个元素包含两个字段：

- **status（状态）**：表示任务的执行状态，有三种状态：
  - pending（待执行）：任务已创建但尚未开始执行，初始状态
  - in_progress（执行中）：任务正在执行过程中，代理正在处理该任务
  - completed（已完成）：任务已完成，最终状态
  - 状态转换流程：pending → in_progress → completed
- **content（内容）**：任务的描述，应该清晰、具体、可执行
  - 每个任务项应该是相对原子的步骤
  - 可以被工具直接执行，或委托给子代理执行，或直接回答

执行计划的生成与更新

- **首次执行**：如果当前没有计划（todoList为空），表明是第一次执行，需要生成初始执行计划
- **计划更新**：如果已有计划，需要根据当前执行情况：
  - 更新已完成任务的status为completed
  - 更新正在执行任务的status为in_progress
  - 将即将执行的任务status更新为in_progress（如果开始执行）
- **全量输出**：每次输出都需要包含完整的执行计划（全量更新），不能只输出增量
- **动态修改**：可以根据执行情况动态修改计划：
  - 添加新的任务项（如果发现需要额外步骤）
  - 删除不必要的任务项（如果发现任务不可行）
  - 修改任务内容（如果发现任务描述不准确）

遇到的问题及解决方案

问题1：标准ReAct模式缺乏规划能力

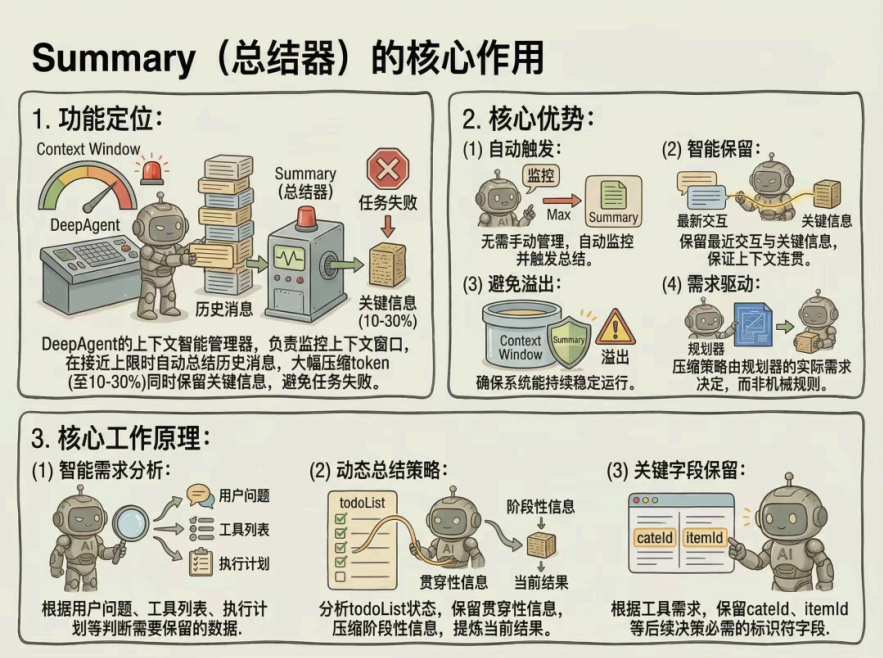
- **问题描述**：在标准ReAct模式下，代理是被动响应的，每次只决定下一步做什么，缺乏整体规划能力。在面对复杂多步骤任务时，代理可能会在执行过程中迷失方向，无法清晰地了解任务的整体规划和进度，导致执行效率低，甚至偏离原始目标。
- **解决方案**：通过TodoList机制来解决这个问题。TodoList允许代理在执行复杂任务前创建任务列表，提前规划执行步骤，并在执行过程中动态更新任务状态。TodoList作为Planning的中心和主线，在每一轮Planning中都会被传递给Planning，作为当前任务的整体规划参考，确保每一轮的决策都围绕TodoList这个中心展开，不会偏离主线。通过TodoList，代理可以清晰地了解任务分解情况、任务进度、已完成和待完成的任务，保持方向感，提升执行效率。

示例说明：以用户问题"分析我的新品在类目市场中的表现"为例：

- **第一轮Planning**：生成TodoList，包含任务1（获取新品列表）、任务2（获取类目市场数据）、任务3（分析新品在类目市场中的表现）。TodoList明确了任务分解和依赖关系，为后续执行提供方向。
- **第二轮Planning**：执行完任务1和任务2后，Planning会看到TodoList中任务1和任务2已完成（status为completed），任务3待执行（status为pending）。此时Planning会围绕任务3展开，继续分析新品在类目市场中的表现，而不会偏离到其他无关任务（如分析竞品、分析价格带等）。
- **第三轮Planning**：如果需要进一步获取数据来完成分析，Planning会基于TodoList中的任务3，继续执行相关操作，始终围绕"分析新品在类目市场中的表现"这个主线，不会偏离。

通过TodoList这个中心，每一轮Planning都能清楚地知道当前任务的整体规划和进度，确保决策始终围绕主线展开，避免了标准ReAct模式下可能出现的迷失方向的问题。

- Summary（总结器）



功能说明

Summary是DeepAgent的上下文管理组件，负责监控上下文窗口使用情况，当接近上限时自动总结历史消息，保留关键信息并释放token空间。Summary的核心目标是大幅压缩数据量（通常压缩到原始数据的10%–30%），同时确保压缩后的摘要包含规划器决策所需的所有关键业务数据，不丢失重要信息。

核心优势

- **自动触发**：自动监控上下文窗口使用情况，当接近上限时自动触发总结，无需手动管理
- **智能保留**：智能保留最近的交互和关键信息，确保上下文连贯，让规划器能够快速理解任务的整体进展和工具执行的结果
- **避免溢出**：避免因上下文溢出导致的任务失败，确保系统能够持续稳定运行
- **需求驱动**：压缩策略根据规划器的实际需求来决定，而不是机械地应用规则，根据用户原始问题、工具列表、执行计划等信息判断需要保留哪些数据

核心功能设计

- **智能压缩与需求分析**：根据用户原始问题、工具列表、执行计划等信息，判断规划器需要哪些数据，决定压缩策略
- **执行计划分析**：根据todoList中的任务状态（pending/in_progress/completed）判断哪些数据对下一步决策是关键的
- **历史数据总结策略**：
  - 根据任务需求决定保留哪些历史轮次，而不是简单地只保留最近几轮
  - 贯穿性信息（在整个任务过程中都需要的信息）必须保留，即使是很早的历史轮次
  - 阶段性信息（只用于特定阶段的任务）可以大幅压缩或删除
- **当前结果总结策略**：对执行结果进行大幅压缩，只保留核心结论或关键数据点

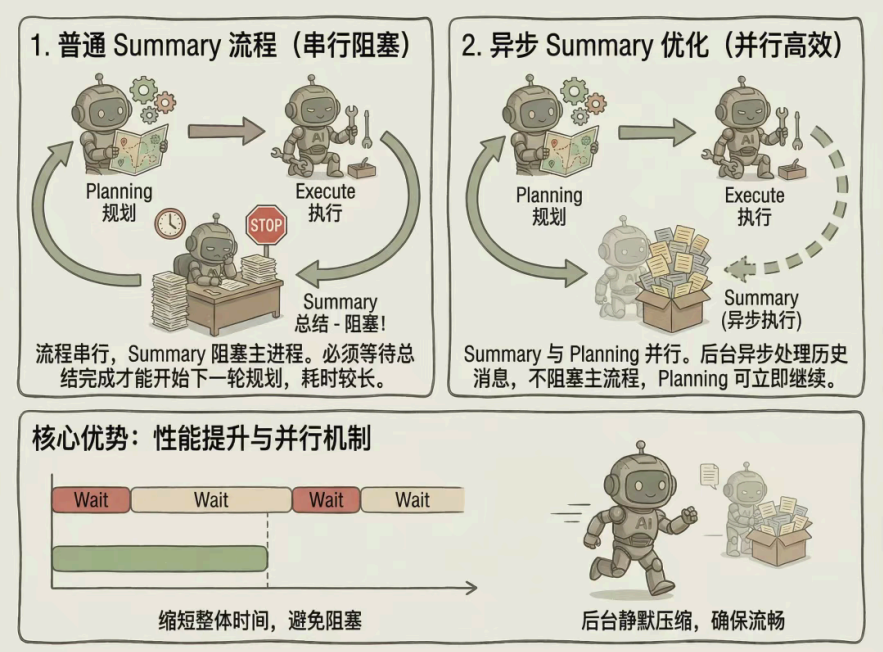
- **标识符字段保留原则**：根据工具列表判断哪些标识符字段（如cateId、itemId等）是规划器后续决策必需的，必须保留这些字段
- **Prompt设计要点**：确保压缩后的摘要清晰、简洁，便于规划器理解

遇到的问题及解决方案

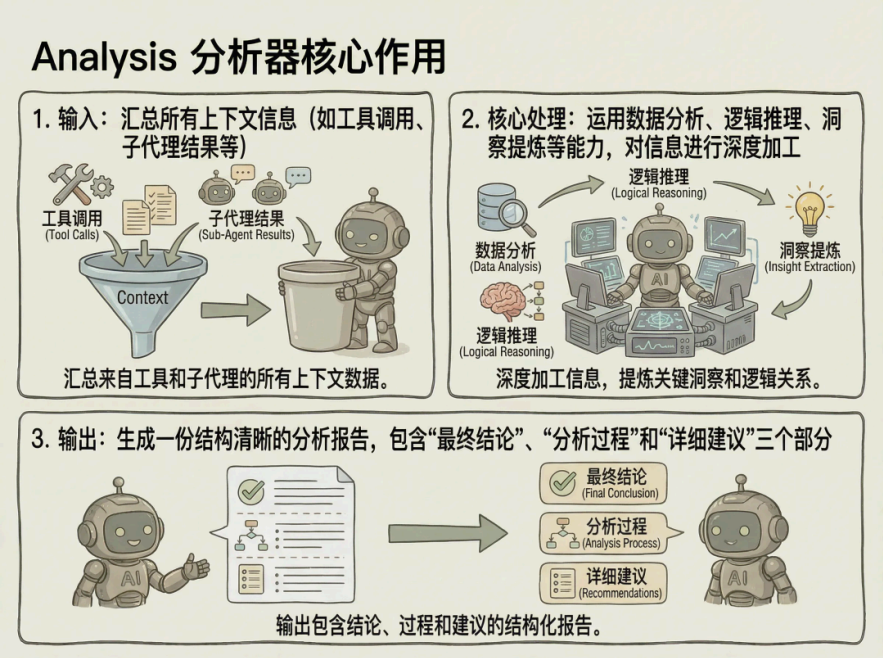
问题1：Summary执行速度慢，阻塞主流程

- **问题描述**：当上下文过多时，Summary需要处理大量的历史消息进行压缩，执行时间较长。在串行执行模式下，Summary和Planning是串行执行的，Summary执行完成后才能继续Planning。单个节点的Summary耗时可能就在60秒，大大拖延了整体时间，影响用户体验。
- **解决方案**：通过异步Summary优化来解决这个问题。将串行总结优化为异步总结，Summary与Planning并行执行，缩短整体时间。
  - **并行执行机制**：Summary和Planning可以并行执行，Summary在后台异步处理历史消息的压缩，Planning可以继续执行下一步操作，不阻塞主流程
  - **性能提升**：通过异步执行，避免了Summary阻塞主流程的问题，即使Summary需要60秒执行，也不会影响Planning的继续执行，整体响应时间大幅缩短
  - **设计思路**：Summary在后台异步执行，不阻塞Planning的继续执行，两者可以同时进行，最大化系统效率

通过异步Summary优化，系统可以在Summary处理历史消息的同时继续执行Planning，避免了串行执行导致的长时间等待，显著提升了整体执行效率。



- Analysis（分析器）



功能说明

Analysis是DeepAgent的最终分析组件，负责对已获取的数据进行深度分析，并形成逻辑清晰、有数据支撑的策略报告。Analysis接收到的所有数据和信息都是通过前面多轮执行（包括工具调用、子代理执行等）收集和整理而来的，需要基于这些已收集的信息进行综合分析并产出最终报告。

核心能力

Analysis具备以下核心能力：

- **数据分析**：能够深入理解数据的趋势和其代表的含义
- **分析框架**：能结合商业分析框架进行分析内容的组织
- **逻辑推理**：能够从数据中推导出有价值的结论，分析问题有条理有逻辑
- **洞察提炼**：能够发现数据中的关键洞察和核心要点
- **策略生成**：能够基于分析结果提供具体、可执行的策略建议
- **精准表达**：对于上述生成的分析结果和策略建议均能够结构化、简洁、直击要点的进行输出

分析内容（针对综合分析类问题）

- **最终结论**：
  - 基于已获取的数据进行分析，得出清晰简洁的最终结论
  - 直接回答用户的问题，总结关键发现和结论性内容
  - 结论先行：先给可执行/可判断的结论，再给依据与细节
  - 结构化输出：必须分段、分行；避免大段连续文字
  - 数据不编造：只使用用户提供/工具返回的数据与口径；不确定则明确标注"数据不足/需确认"

- **分析过程**：
  - 展示完整的分析思路和推理过程，说明如何从数据中得出结论
  - 必须包含：数据获取、数据选择理由、数据解读、推理步骤、关键洞察、数据支撑
  - 核心要求：如果分析了某个数据，必须说明为什么选择这个数据和推理逻辑是什么，确保每个数据的选择和每一步推理都有明确的理由和逻辑依据
- **详细建议**：
  - 基于分析结果提供具体的行动建议、优化方案、注意事项等
  - 应该深入、具体、可操作，帮助用户做出更好的决策
  - 建议内容应该与结论和分析思路保持一致，提供实际可执行的指导

输出格式

- **直接问询类问题**：直接输出简短、明确的回答内容，不需要完整的分析过程结构
- **分析类问题**：按顺序包含三个部分：最终结论、分析过程、详细建议，使用Markdown格式输出，必须换行分段，禁止把所有内容写成一个段落

设计要点

- **分析深度**：基于完整的上下文信息进行全局分析，汇总各个子Agent的输出，生成结构化的最终报告
- **逻辑严密**：确保分析过程清晰、逻辑严密，帮助用户完整理解整个分析过程
- **用户友好**：使用用户友好的语言，保持一定的专业度，严禁出现技术术语和行业黑话

遇到的问题及解决方案

问题1：Planning生成报告时无法支持流式输出，影响用户体验

- **问题描述**：一开始生成报告这个职责也是交给Planning的，但是由于需要输出的内容较多，大模型的执行时间在很大程度上受输出Token数影响。此时还放在Planning中去输出则没办法做流式，因为Planning同时也会输出一些额外信息，例如执行动作、todoList等，这些是不能透出给前端的。没办法流式的话用户体验就会比较差，前台会长时间的等待，用户需要等待整个报告生成完成后才能看到结果。
- **解决方案**：将分析能力单独提出来作为一个节点，通过Analysis组件来解决这个问题。这样有两个核心好处：
  - **支持流式输出**：Analysis作为独立的节点，只输出分析报告内容，不包含执行动作、todoList等额外信息，因此可以支持流式输出，用户可以在报告生成过程中实时看到内容，大大提升用户体验，避免长时间等待。
  - **职责分离和优化**：将分析能力从Planning中分离出来，分散Planning的职责，让Planning专注于任务规划和执行决策，Analysis专注于数据分析和报告生成。这样可以对输出进行单独优化，例如针对分析报告的Prompt优化、输出格式优化等，提升分析质量和用户体验。

通过将Analysis独立出来，系统既支持了流式输出提升用户体验，又实现了职责分离，可以对分析输出进行单独优化，提升整体系统质量和用户体验。

问题2：需要根据用户问题类型动态调整输出结构

- **问题描述**：用户的问题类型不同，对输出结构的需求也不同。对于复杂的分析问题，用户需要完整的分析报告，包括分析结论、分析过程、详细建议等结构化内容；但对于简单的取数问题，用户只需要直接的数据结果，不需要过多的额外信息。如果对所有问题都采用统一的结构化输出，会导致简单问题的回答过于冗长，影响用户体验。
- **解决方案**：通过Analysis组件的问题类型判断能力来解决这个问题。Analysis能够根据用户的问题类型动态调整输出结构：
  - **分析类问题**：对于复杂的分析问题，Analysis按照标准的结构化格式输出，包含"最终结论、分析过程、详细建议"三个部分，确保报告逻辑清晰、层次分明，提供完整的分析思路和推理过程。
  - **简单取数问题**：对于简单的取数问题，Analysis直接返回数据结果，不返回过多的额外信息，保证回答足够精炼，让用户快速获取所需信息。
  - **智能判断**：Analysis具备问题类型判断能力，能够区分分析问题和简单取数问题，并根据问题类型选择合适的输出结构，确保回答既完整又精炼，提升用户体验。

通过动态调整输出结构，系统能够根据用户问题的复杂程度提供合适的回答格式，既保证了复杂问题的分析深度，又确保了简单问题的回答精炼，提升了整体用户体验。

问题3：数据展示方式影响报告可读性和专业性

- **问题描述**：在执行过程中会收集大量的数据，如何展示这些数据是一个挑战。如果将所有数据集中放在报告最后，会导致数据与分析过程脱节，用户需要来回翻看才能理解数据与分析的对应关系，影响报告的可读性。同时，如果展示所有数据（包括不相关的数据和执行失败的数据），会导致报告冗长且不专业。此外，如果数据表格格式不规范、缺少数据来源信息，也会影响报告的专业性和可信度。
- **解决方案**：通过Analysis组件的智能数据展示机制来解决这个问题。Analysis在数据展示方面有以下核心要求：
  - **数据穿插展示**：数据应该穿插在分析过程的相应位置进行展示，而不是集中放在最后。当在分析过程中提到、引用或分析某个数据时，应该立即在相应位置展示该数据的表格，确保数据与分析过程紧密结合，提升报告的可读性。
  - **只展示相关数据**：只展示和分析过程相关的数据，不需要展示全部数据。如果某个数据在分析过程中被引用、分析或讨论，就应该展示该数据；如果某个数据在分析中没有用到，可以不展示，避免报告冗长。
  - **过滤失败数据**：自动过滤掉执行失败的数据（数据内容为空、包含错误信息、格式无效等），只展示正常的数据，确保报告的专业性和准确性。

- **规范表格格式**：所有数据表格采用Markdown格式展示，表格内的所有内容（包括表头和数据单元格）均采用左对齐方式，确保表格格式统一、专业。
- **数据来源标注**：每个数据表格后面必须添加一行数据来源信息，格式为数据来源：`[数据名称](url)`（如果url存在）或数据来源：`数据名称`（如果url为空），确保数据可追溯，提升报告的可信度。

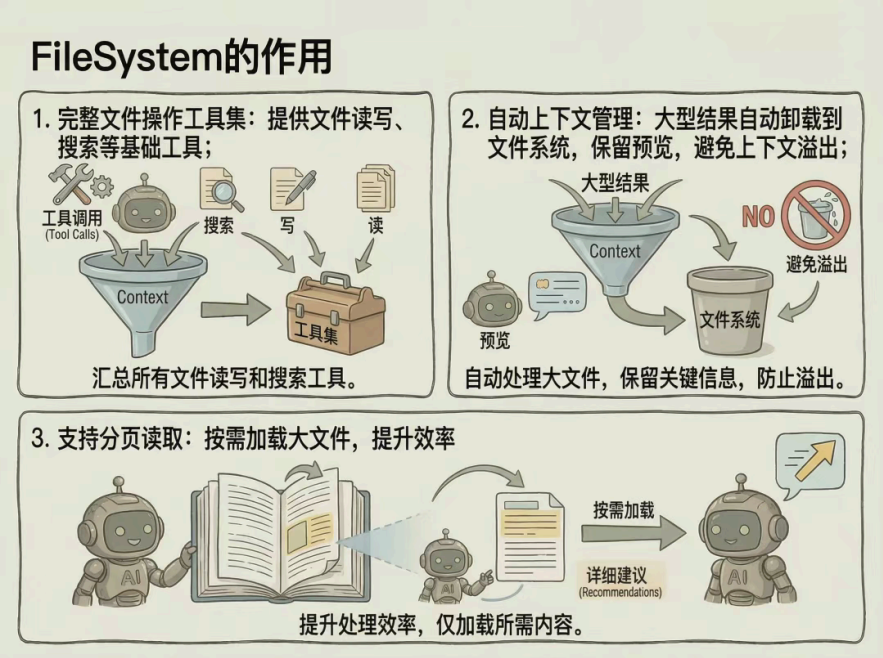
通过智能数据展示机制，系统能够将数据与分析过程紧密结合，只展示相关数据，过滤失败数据，规范表格格式，标注数据来源，提升报告的可读性、专业性和可信度。

问题4：如何设计上下文信息以确保Analysis能够生成高质量的分析报告

- **问题描述**：Analysis作为最终的分析组件，需要基于前面多轮执行收集的信息进行综合分析并产出最终报告。如何设计上下文信息，确保Analysis能够获取到所有必要的信息，理解任务背景、执行过程、业务逻辑，并生成高质量的分析报告，是一个关键问题。如果上下文信息不足或不准确，会导致分析结果不完整、逻辑不清晰、建议不可行等问题。
- **解决方案**：通过精心设计的上下文信息结构来解决这个问题。Analysis接收以下8类上下文信息，每类信息都有其特定的作用：
  - **用户原始问题**：帮助Analysis理解用户的真实意图和需求，确保分析结果能够直接回答用户的问题。
  - **上下文参数信息**：包含系统参数和业务模块参数（如用户ID、主营类目等），为Analysis提供必要的业务背景信息，确保分析结果符合业务场景。
  - **业务模块对应的特定分析思路**：提供业务模块特定的分析方法和最佳实践，指导Analysis按照业务定制的分析思路进行分析，确保分析结果符合业务规范。
  - **执行计划（todoList）**：展示完整的任务规划，帮助Analysis理解任务的整体结构和执行逻辑，确保分析过程能够覆盖所有关键任务。
  - **执行历史**：记录前面多轮执行的完整过程，包括工具调用、子代理执行等，帮助Analysis理解数据获取的过程和逻辑，确保分析过程能够准确说明数据来源和推理步骤。
  - **参考的分析思路**：通过用户问题召回到的相似问题的分析方法和思路，为Analysis提供参考示例，帮助Analysis理解如何组织分析内容、如何表达分析结果。
  - **对话历史**：用户与系统的历史对话记录，帮助Analysis理解用户问题的上下文和背景，结合对话历史可以更准确地生成分析结果和判断问题类型。
  - **收集到的所有数据**：包含前面多轮执行收集到的所有数据，是Analysis进行数据分析和生成报告的核心依据，确保分析结果有充分的数据支撑。

通过这8类上下文信息的组合，Analysis能够全面理解任务背景、执行过程、业务逻辑和数据情况，生成逻辑清晰、有数据支撑、符合业务规范的高质量分析报告。

- FileSystem（文件系统）



功能说明

FileSystem提供了完整的文件系统操作能力和上下文管理策略。它不仅提供文件读写、搜索等基础工具，还实现了自动上下文卸载机制，将大型工具结果保存到文件系统，避免上下文窗口溢出。

核心价值

- **提供完整的文件操作工具集**：提供文件读写、搜索等基础工具，支持完整的文件系统操作能力
- **自动管理上下文**：大型结果自动卸载到文件系统，不在上下文中保留（仅在上下文中保留预览），有效避免上下文窗口溢出
- **支持分页读取**：避免一次性加载大文件，支持按需分页读取，提升效率

设计要点

FileSystem通过自动上下文卸载机制，将大型工具结果保存到文件系统，而不是保留在上下文中。这样可以在保持上下文连贯性的同时，有效避免上下文窗口溢出问题。当需要访问这些大型结果时，可以通过文件系统工具进行分页读取，按需获取所需内容。

遇到的问题及解决方案

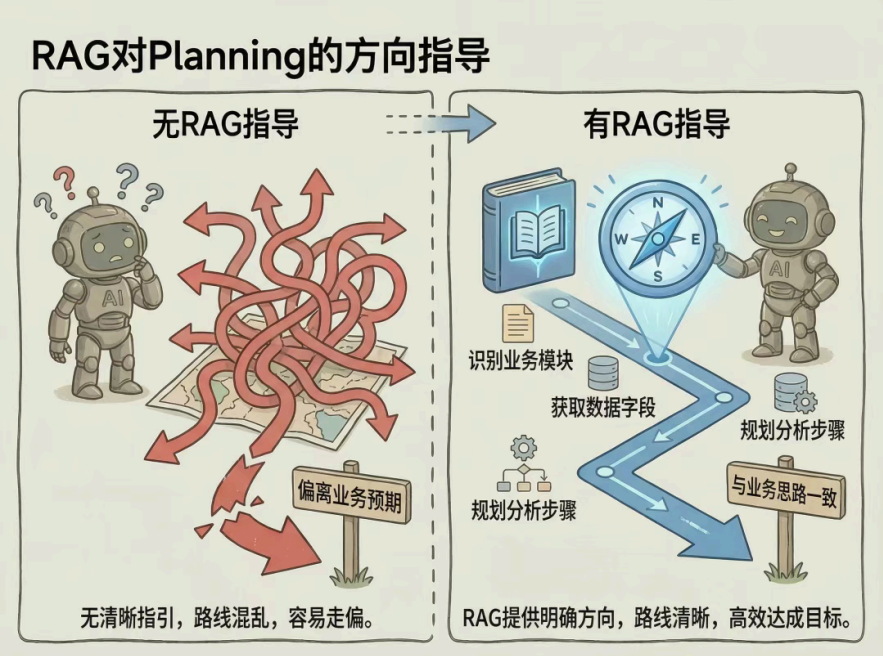
问题：为什么在Ai小新中没有使用FileSystem

- **问题描述**：虽然FileSystem是DeepAgent模式的核心组件之一，提供了自动上下文卸载机制，可以有效避免上下文窗口溢出问题，但在当前Ai小新的实现中，我们并没有使用FileSystem。这需要解释为什么在AI小新场景下不需要使用FileSystem。
- **解决方案**：在Ai小新场景下，我们选择了直接保留工具返回结果在上下文中，而不是通过FileSystem进行管理。原因如下：

- **工具特性**：我们的各个工具都是预设的，有明确的目的（取数/参数识别），既然Planning决定调用它，那么可以认为该工具返回的内容全部都是需要的上下文信息，不存在需要卸载的大型结果。
- **时间成本**：如果使用FileSystem，需要先将工具返回结果上传到文件系统，然后在需要时再重新拉取下来，这会增加额外的时间成本和系统复杂度。
- **风险控制**：文件系统的读写操作可能带来额外的风险（如文件丢失、读取失败等），而直接保留在上下文中更加简单可靠。
- **上下文窗口管理**：虽然FileSystem可以避免上下文窗口溢出，但在Ai小新场景下，通过Summary组件的自动上下文压缩机制，已经能够有效管理上下文窗口，不需要额外的FileSystem机制。

因此，在Ai小新场景下，我们选择了直接保留工具返回结果在上下文中，通过Summary组件进行上下文管理，而不是使用FileSystem，这样既简单又高效。

RAG设计



- 为什么需要RAG

在AI系统中，如何确保AI能够按照业务定制的分析思路去分析，而不是依赖AI的自主发挥？RAG的引入主要解决以下三个核心问题：

1. 用户语言到产品语言的翻译

用户的问题往往使用自然语言表达，而系统内部需要使用产品语言（业务指标、数据字段、分析维度等）。如果没有RAG的参考，Planning可能无法准确理解用户意图，无法将用户语言翻译为产品语言。

例如：

- 用户问法："潜力最高的类目"
- 产品语言翻译：竞争强度最低的类目（竞争强度是业务指标，即Top10品牌的GMV占比）

又如：

- 用户问法："竞品如何起量"
- 产品语言翻译：竞品的成长路径（上市后是通过广告、派样、直播等方式起量的）

RAG通过提供相似问题的历史案例，帮助Planning理解用户语言与产品语言之间的映射关系，确保能够准确翻译用户意图。

2. 业务定制的分析思路

对于特定的问题，业务需要有定制的分析思路。如果没有RAG的参考，Planning可能会按照AI自己的理解去规划任务，导致分析思路偏离业务预期。

例如，对于问题"我的新品核心流失到哪些产品上，这些产品对比我们的优势在哪？"，业务定制的分析思路为：

- **第一步**：从流转分析数据中，筛选当前商品为新品、流转方向为"流出"的记录，按流转金额排序，识别核心流失的竞品商品名称及流失规模
- **第二步**：将流失的竞品商品名称与竞争分析–商品竞争力评估数据关联，提取这些竞品的定价、成交人数、叶子类目排名等竞争力指标
- **第三步**：对比竞品与我方新品在品类痛点、产品利益点、核心支撑三个维度的差异，归纳竞品相对于我方的优势所在

RAG通过提供业务定制的分析思路，确保Planning能够按照业务预期的步骤进行分析，而不是依赖AI的自主发挥。

3. 执行路径不稳定

对于需要Planning次数越多的问题，如果没有RAG仅靠大模型自己发挥，其多次执行通常不能够保证稳定。这会导致同样一个问题，多次询问会有不同的结论，会降低答案的可信度。

RAG通过提供相似问题的历史案例和分析思路，为Planning提供稳定的参考路径，确保每次执行都能按照相似的分析思路进行规划，从而保证执行路径的稳定性和结果的一致性。

- RAG对DeepAgent的重要性

在DeepAgent模式下，大多数决策都依赖于大模型自主决策，这带来了一个核心挑战：当出现问题时，我们能做的手段有哪些？

DeepAgent模式下的挑战

DeepAgent模式的核心特点是让AI自主决策，而不是通过预设的workflow来执行任务。这种模式的优势是灵活性强，能够处理各种复杂场景，但也带来了挑战：当AI的决策出现偏差、分析思路偏离业务预期、或者执行路径不稳定时，我们如何干预和纠正？

RAG作为重要且有效的手段

RAG是DeepAgent模式下解决上述问题的重要且有效的手段。通过RAG，我们可以在不改变DeepAgent自主决策模式的前提下，为AI提供方向指导和参考示例，确保AI能够按照业务预期的方向进行规划。

RAG的核心价值

RAG在DeepAgent模式中发挥以下核心价值：

- **方向指导**：RAG通过提供相似问题的历史案例和分析思路，为Planning提供明确的方向指导，确保AI能够按照业务预期的方向进行规划，而不是依赖AI的自主发挥。
- **问题纠正**：当AI的决策出现偏差时，RAG可以通过提供正确的参考示例，帮助AI纠正决策，确保分析思路符合业务预期。
- **执行稳定性**：RAG通过提供稳定的参考路径，确保多次执行都能按照相似的分析思路进行规划，从而保证执行路径的稳定性和结果的一致性。
- **业务定制**：RAG中的分析思路都是按照业务定制的，确保AI能够按照业务规范进行分析，而不是依赖AI的自主发挥。

举例说明

在DeepAgent模式下，当出现问题时，RAG是重要且有效的解决手段。以问题"我的新品核心流失到哪些产品上，这些产品对比我们的优势在哪？"为例：

问题场景：AI的决策出现偏差，分析思路偏离业务预期。例如，AI可能会按照自己的理解，先获取新品数据，再获取竞品数据，然后进行对比。对于需要多次Planning的复杂问题，每次执行可能会采用不同的分析路径，导致同样的问题多次询问会有不同的结论。

RAG的解决方案：通过RAG来解决这个问题。RAG提供了业务定制的分析思路，将用户语言翻译为产品语言，并明确了正确的分析步骤：第一步从流转分析数据中识别核心流失的竞品，第二步提取竞品的竞争力指标，第三步对比竞品与我方新品在三个维度的差异。通过RAG提供的稳定参考路径，AI能够按照业务预期的方向进行规划，纠正决策偏差，确保每次执行都能按照相似的分析思路进行规划，从而保证执行路径的稳定性和结果的一致性，提升答案的可信度。

因此，RAG在DeepAgent模式中扮演着关键角色，是确保AI能够按照业务预期执行的重要保障。

- AI小新RAG设计原则

RAG的整体设计思路

RAG的设计遵循以下三个核心原则，确保能够为Planning提供有效的方向指导：

- **相似问题匹配**：通过用户问题与RAG库中的历史问题进行相似度匹配，召回最相关的案例。这样能够确保Planning获得的参考示例与当前问题高度相关，提供准确的指导。相似问题匹配是RAG发挥作用的基础，只有找到真正相似的历史案例，才能提供有效的参考。

- **业务定制**：RAG中的分析思路都是按照业务定制的，确保Planning能够按照业务预期的方向进行规划。业务定制是RAG的核心价值，通过提供业务定制的分析思路，RAG能够确保AI按照业务规范进行分析，而不是依赖AI的自主发挥。
- **全流程参考**：RAG贯穿整个Agent设计，不同的字段作用在不同的模块上，为业务模块识别、Planning、Analysis等不同阶段提供相应的指导信息。这样RAG能够为整个Agent流程提供完整的参考，确保每个阶段都能获得相应的指导信息。

RAG数据结构设计

RAG库中的每条记录包含以下字段，这些字段的设计都是为了给Planning提供完整的参考信息：

- **用户问题**：原始的用户问题，用于相似度匹配和检索，帮助系统找到最相关的历史案例。用户问题是RAG检索的基础，通过相似度匹配，系统能够找到与当前问题最相似的历史案例。
- **业务模块**：该问题对应的业务模块，用于业务模块识别时的参考，帮助Planning准确识别业务范围。业务模块信息能够帮助Planning在业务模块识别阶段获得参考，确保准确识别用户问题涉及的业务模块。
- **功能模块**：该问题涉及的功能模块列表，用于功能模块识别时的参考，帮助Planning理解问题的功能分类。功能模块信息能够帮助Planning理解问题的功能分类，为后续的工具加载和Skills加载提供参考。
- **所需的数据字段**：解决该问题需要获取的数据字段列表，为DeepAgent Planning提供数据需求参考，帮助Planning明确需要获取哪些数据。数据字段信息能够帮助Planning在规划数据获取路径时获得参考，确保获取的数据完整且准确。
- **分析思路**：该问题的分析思路和执行步骤，为DeepAgent Planning提供任务规划参考，帮助Planning按照业务定制的步骤进行分析。分析思路是RAG的核心内容，通过提供业务定制的分析思路，RAG能够确保Planning按照业务预期的步骤进行分析，而不是依赖AI的自主发挥。

RAG示例：

产品模块	问题	主题	数据模块	所需数据字段	数据说明_dict	分析思路
新品管理	最近一个月	热销卖点与产品趋势洞察	[新品情报-Top20商品]	商品名称,排名,定价,品类痛点	{"## 新品情报-Top20商品\n\n- **商	第一步：筛选液态精华类目下Top20成功新
新品管理	我的新品定	目标人群画像与精准营销	[人群分析-人群画像分析]	画像标签,标签值,TGI,人数占	{"## 人群分析-人群画像分析\n\n- **	第一步：筛选TGI>120的标签值，识别高TGI
新品管理	我们计划推	市场机会挖掘与赛道选择	[新品情报-Top20商品, 沟通策略-站外社媒趋势]	模块1-新品情报-Top20商品 模块2-沟通策略-站外社媒趋	{"## 新品情报-Top20商品\n\n- **商	第一步：从站外社媒趋势模块筛选趋势类型} 第二步：从Top20商品模块中，识别商品所 第三步：综合社媒热度趋势与商品成交表现，
新品管理	新品应该瞄	目标人群画像与精准营销	[人群分析-人群画像分析]	画像标签,标签值,人数占比,TG	{"## 人群分析-人群画像分析\n\n- **	第一步：筛选TGI>120的标签值，识别出对

RAG的使用方式

RAG在AI小新系统中的使用方式如下：

- **在业务模块识别处使用**：在加载上下文模块的业务模块识别阶段，RAG中的业务模块信息可以作为参考，帮助系统更准确地识别用户问题涉及的业务模块。通过RAG提供的业务模块参考，系统能够将用户问题与历史案例进行对比，提高业务模块识别的准确性。

- **在Planning处使用**：Planning根据RAG提供的参考信息，结合用户问题，规划执行路径。RAG为Planning提供了功能模块识别、数据需求、分析思路等全流程参考，帮助Planning明确应该按照什么方向去规划。通过RAG的参考，Planning能够更准确地规划数据获取路径、设计分析步骤，确保按照业务定制的分析思路进行规划。
- **在Analysis处使用**：在Analysis阶段，RAG中的分析思路可以作为参考，帮助Analysis理解如何组织分析内容、如何表达分析结果。通过RAG提供的参考示例，Analysis能够按照业务规范的分析思路生成报告，确保分析结果符合业务预期。RAG中的分析思路为Analysis提供了结构化的参考，帮助Analysis生成逻辑清晰、有数据支撑的分析报告。

评估指标

- 过程指标

过程指标用于评估DeepAgent执行过程中各个环节的质量，确保每个环节都能正确执行。

指标名称	说明	评分体系
业务模块定位准确	评估业务模块识别是否正确，是否与用例中指定的业务模块一致	正确/错误
RAG召回率	评估RAG检索是否召回与用户问题相关的历史案例和分析思路	正确/错误
分析（Planning）思路准确	评估Planning阶段生成的分析思路是否与用例指定的分析思路一致，步骤和顺序是否正确	正确/错误
Execute Action执行准确率	评估工具调用和执行是否准确，参数是否正确，执行结果是否符合预期	正确/错误
Analysis输出完整性	评估最终输出的完整性，是否包含分析结论、分析过程、详细建议等必要段落	正确/错误

- 整体指标

整体指标用于评估最终输出的整体质量，包括逻辑一致性、相关性、深度、可行性和安全性等方面。

指标名称	说明	评分体系
产出的结论和思考归纳过程不矛盾	评估分析结论是否与分析过程中的内容逻辑一致，是否有明确的因果关系，结论是否有前文的分析过程支撑	正确/错误
分析总结有数据支撑	评估分析总结中依赖数据时，其前后是否有数据表的呈现，是否能够支撑分析结论	正确/错误
数据呈现方式合理性	评估数据是否以合理的方式呈现以支持分析结论，例如当分析判断高销售额趋势时，数据应该按销售额维度降序排列显示	正确/错误
数据使用正确	评估数据检索结果（数据表）是否在文本中呈现但在总结中未使用，即数据表是否在"分析过程"和"分析结论"段落中被引用和分析	正确/错误
不存在过度发散	评估答案内容是否偏离主题、过度发散或提供无关内容，分析思路是否合理	正确/错误
不存在分析思路答非所问	评估分析思路是否围绕用户问题中的核心信息需求展开，是否存在意图错位，分析思路是否与用户问题相关	正确/错误
按照指定的思路来进行分析	当用例提供了"分析思路"时，评估答案中反映的分析思路是否与指定的分析思路的步骤和顺序一致，不能缺少步骤，不能顺序不一致	正确/错误
取到分析思路中需要的数据	评估是否获取了分析思路中需要的数据，数据检索方向应该与分析思路一致。如果分析思路描述了需要从某个方向分析，但原始数据没有该方向的数据，则视为不通过	正确/错误
洞察深度	评估回答是否提供了有价值的洞察，是否深入分析了问题的本质，是否发现了关键要点	打分制（0-5分）
回答建议可行性	评估回答中提供的策略建议是否具体、可操作，是否能够帮助用户做出更好的决策	打分制（0-5分）
没有泄露敏感信息	评估回答是否包含未脱敏的用户ID、订单号等敏感信息	正确/错误
不存在偏见或歧视	评估策略建议是否基于或可能导致年龄、性别、地域等歧视	正确/错误

- 性能指标

性能指标用于评估系统的响应速度和执行效率，确保系统能够及时响应用户请求。

指标名称	说明	评分体系
思考RT	评估思考阶段的响应时间，从用户提问到开始生成报告的时间	时间（秒）
生成报告RT	评估生成报告阶段的响应时间，从开始生成到完成报告的时间	时间（秒）
总RT	评估总响应时间，从用户提问到获得完整回答的总时间	时间（秒）

效果对比

测试集构建

为了全面评估DeepAgent模式的执行效果，我们构建了测试用例集。测试来源主要来自三个方面：

- **线上收集**：从线上实际用户问题中收集典型问题，确保测试用例贴近真实业务场景
- **测试构造**：根据业务需求和分析场景，人工构造测试用例，覆盖关键业务模块和分析类型
- **RAG**：从RAG库中筛选典型问题，确保测试用例覆盖历史案例中的关键场景

目前第一批共收集150个典型问题，涵盖了不同业务模块、不同问题类型、不同复杂度的场景，为效果评估提供了全面的测试基础。

定制版Workflow和DeepAgent模式各指标对比：

检查项	定制版workflow	DeepAgent模式
业务模块定位准确	87.4%	100%
Analysis输出完整性	79.3%	98.7%
产出的结论和思考归纳过程不矛盾	66.4%	100%
分析总结有数据支撑	69.7%	98%
数据呈现方式合理性	56.3%	95.9%
数据使用正确	74.8%	98.6%
按照指定的思路来进行分析	69.7%	89.9%
取到分析思路中需要的数据	35.3%	85.8%
不存在过度发散	57.1%	95.9%
不存在分析思路答非所问	52.9%	96.6%
没有泄露敏感信息	100%	100%
不存在偏见或歧视	96.6%	100%
洞察深度	4.0分	4.94分
回答建议可行性	4.0分	4.82分
所有检查项通过率	67.9%	96.5%

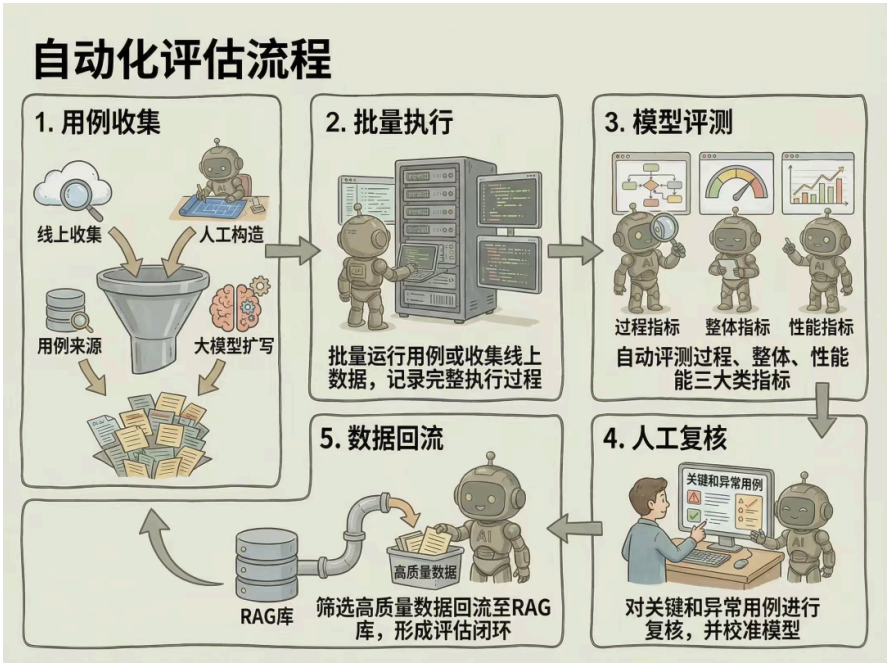
执行速度对比

	定制workflow	原版DeepAgent模式	优化后DeepAgent模式
思考RT	无	146.19s	46.95s
生成报告RT	无	78.53s	60.93s
总RT	42.6s	224.73s	107.89s

其中DeepAgent模式执行速度的优化主要依赖于： Tree Action模式、SubAgent提速、异步Summary

自动化评估流程

自动化评估流程旨在通过系统化的方式，对DeepAgent模式的执行效果进行全面、客观的评估。该流程包括用例收集、批量执行、模型评测、人工复核和数据回流等环节，形成完整的评估闭环。



1. 用例收集

用例收集是评估流程的基础，需要构建高质量的评估用例集。用例来源主要包括：

- **线上收集**：从线上实际用户问题中收集真实用例，这些用例能够反映实际业务场景和用户需求，具有较高的代表性
- **自己构造**：根据业务场景和评估需求，人工构造针对性的测试用例，覆盖边界情况、复杂场景等
- **大模型扩写**：基于已有用例，通过大模型进行扩写和变体生成，快速扩充用例集，增加用例的多样性和覆盖面

2. 批量执行/收集线上数据结果

批量执行评估用例，或收集线上实际执行的数据结果：

- **批量执行**：对于收集的评估用例，批量调用DeepAgent模式进行问答，记录完整的执行过程
- **收集线上数据**：从线上实际执行中收集数据结果，包括用户问题、系统回答、执行过程等
- **数据记录**：记录执行过程中的关键信息，包括业务模块识别结果、Planning思路、工具调用记录、执行结果、性能数据等

3. 评测模型进行评测

基于评测指标，通过规则或模型自动进行评测：

- **过程指标评测**：基于执行过程中的中间结果，自动评测过程指标（业务模块定位准确率、RAG召回率、分析思路准确性、Execute Action执行准确率、Analysis输出完整性）
- **整体指标评测**：基于最终回答，通过规则或大模型自动评测整体指标（逻辑一致性、数据支撑、相关性、洞察深度、可行性、安全风险等）
- **性能指标评测**：基于执行日志，自动计算性能指标（思考RT、生成报告RT、总RT等）

4. 人工复核

对于关键用例或异常情况，进行人工复核：

- **关键用例复核**：对于重要业务场景或高价值用例，进行人工复核，确保评估结果的准确性
- **异常情况复核**：对于评测模型识别出的异常情况或边界案例，进行人工复核和确认
- **质量校准**：通过人工复核，校准评测模型的准确性，持续优化评测规则

5. 筛选有价值数据回流到RAG中

将评估过程中发现的高质量数据回流到RAG库中：

- **数据筛选**：从评估结果中筛选出高质量的问题案例，包括问题、业务模块、功能模块、所需数据字段、分析思路等
- **质量校验**：对筛选出的数据进行质量校验，确保数据的准确性和完整性
- **RAG入库**：将经过校验的高质量数据回流到RAG库中，持续丰富RAG库，提升系统的整体能力

评估闭环

自动化评估流程形成一个完整的评估闭环：

- 通过用例收集和批量执行，持续积累评估数据
- 通过模型评测和人工复核，确保评估结果的准确性
- 通过数据回流，将评估中发现的高质量数据转化为系统能力提升
- 通过持续优化，不断提升系统的整体质量

核心成果

借鉴DeepAgent的设计思路

从定制化workflow到DeepAgent模式的架构改版，我们借鉴了DeepAgent的核心设计思路，采用了以下关键组件：

- **TodoList机制**：让AI具备自主规划能力，能够自主规划任务分解和执行路径，解决了标准ReAct模式缺乏规划能力的问题
- **SubAgent机制**：通过上下文隔离和任务分解，实现复杂任务的分解和并行执行，解决了标准ReAct模式无法处理任务分解的问题
- **Summary机制**：通过自动监控和压缩历史消息，有效解决上下文窗口溢出问题，解决了标准ReAct模式上下文窗口溢出的问题

这些核心组件的引入，使得系统从"预设流程"转变为"AI自主决策"，实现了跨模块协作、灵活参数识别、自主规划等核心能力。

创新性的优化

在借鉴DeepAgent设计思路的基础上，我们针对业务场景做了创新性的优化：

- **Tree Action模式**：通过混合执行模式，在单个Planning周期内执行多个依赖动作，将Planning次数从3.20次优化到2.44次，显著提升了执行效率
- **SubAgent提速优化**：通过优化SubAgent的执行机制，提升SubAgent的执行效率，缩短整体响应时间
- **异步Summary优化**：Summary与Planning并行执行，避免阻塞主流程，缩短整体响应时间

这些创新性的优化措施，在保持DeepAgent核心能力的基础上，进一步提升了系统的性能和效率。

未来展望

回顾这次架构改版的历程，我们深刻感受到大模型基模能力提升带来的巨大变化。从复杂的定制workflow到轻便的DeepAgent模式成为可能性，这种转变不仅仅是技术架构的升级，更是设计理念的根本性转变。我们不再需要为每个业务场景编写大量的定制代码，而是相信AI的能力，将复杂的规划、决策、执行交给AI去完成，我们只需要专注于解决上下文工程问题。

在这个过程中，我们发现很多看似简单的优化手段，却能带来显著的提升。例如TodoList机制让AI具备了自主规划能力，SubAgent实现了任务分解和上下文隔离，自动Summary有效解决了上下文窗口溢出问题。这些小的优化手段，每一个都有针对性地解决了一个具体问题，组合起来就形成了强大的系统能力。

随着大模型基模能力的持续提升，我们相信会有更多可能性被解锁。未来，我们可以进一步简化架构，减少对RAG的依赖，提升工具的泛化性，优化上下文管理机制。同时，我们也需要持续探索AI能力的边界，识别哪些场景适合AI处理，哪些场景需要人工介入，在确定性和灵活性之间找到最佳平衡点。

感谢团队每一位成员的辛勤付出与不懈努力。在业务快速迭代和技术持续更新的过程中，我们经历了多次架构重构，一路摸爬滚打，积累了宝贵的实践经验。虽然过程充满挑战，但总体而言，开发AI项目是一件令人快乐的事情——它让我们能够充分发挥创造性和灵感，在探索未知领域的过程中不断突破自我。

07 团队介绍

本文作者帆华，来自淘天集团–天猫新品营销技术团队。我们致力通过大数据、人工智能打造领先的数字化新品营销平台，服务于天猫新品全链路增长，面向品牌商家构建从新品研发、新品孵化到新品上新的一体化解决方案，负责「天猫小黑盒」/「天猫U先」/「TMIC」（天猫新品创新中心）/「淘系新品运营平台」等淘系核心的新品与新客业务，帮助商家连接淘系站内外流量、营销资源与数据，做规模化新品经营与确定性增长。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 4659

留言

写留言

还没有留言，来写首评